KTH

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING
SECOND CYCLE, 30 CREDITS

Stockholm, Sweden 2026

Joint Radar Emitter and Mode Clustering Using Transformer Encoder Architecture

Markus Johnson Swegmark

Supervisors: Saikat Chatterjee (KTH)
Alexander Karlsson (Saab)
Elin Ohlman (Saab)
Examiner: Magnus Jansson
Host: Saab AB

KTH Royal Institute of Technology
School of Electrical Engineering and Computer Science
Master's Programme in Machine Learning

TRITA-EECS-EX-XXXX:XXX

Abstract

Passive electronic support measures record a time-ordered stream of pulse descriptor words in which several radars are interleaved and each radar may change operating mode. Operators need both groupings from the same stream. Classical pipelines deinterleave first and then classify modes. That split does not produce one representation for both partitions, and it assumes emitter names that can be compared across recordings.

This thesis trains a transformer encoder with a shared trunk and two branches on synthetic, fully labelled windows of length 2000. Each pulse has a recording-local emitter identity and a mode drawn from a catalogue of 16 types shared across recordings. Two supervised contrastive losses, identical in form, differ only in the positive set: emitters within a recording, modes across the batch. At inference, DBSCAN clusters each branch independently. No catalogue softmax is used.

On 960 non-overlapping test windows, standard self-attention already recovers both partitions on most windows (mean emitter ARI 0.944, mode ARI 0.986). Rotary encoding of time of arrival, and a pairwise physical attention bias, raise emitter ARI to 1.000 and remove the remaining failure tail. Rotary encoding of planar incidence on the deinterleaving branch helps less. The bias matches the time-rotary scores and spends extra memory and encoder time on pairwise maps of size $L \times L$. Using both mechanisms together lowers emitter ARI to 0.853.

A Vanilla encoder trained only for deinterleaving outperforms the joint model on emitters (ARI 0.999 against 0.944) while its untrained mode branch merges. A Vanilla encoder trained only for modes matches the joint mode scores (ARI 0.984) while its untrained emitter branch splits. The joint objective is therefore a trade-off, not a free extra head. Held-out modulations and operational recordings are not scored.

Keywords: transformer, deep clustering, electronic intelligence, radar emitter identification, mode recognition, supervised contrastive learning.

Sammanfattning

Passiv signalspaning ger en tidsordnad ström av pulsbeskrivande ord där flera radarstationer är sammanflätade och varje station kan byta arbetsmod. Båda grupperingarna behövs ur samma ström. Klassiska kedjor avinterfolierar först och klassificerar sedan moder. Den uppdelningen ger inte en gemensam representation, och den förutsätter sändarnamn som kan jämföras mellan inspelningar.

Detta arbete tränar en transformerkodare med en gemensam stam och två grenar på syntetiska, fullt märkta fönster av längd 2000. Varje puls har en sändaridentitet som bara gäller inom inspelningen och en mod ur en katalog med 16 typer som delas mellan inspelningar. Två övervakade kontrastiva förluster, identiska till formen, skiljer sig bara i den positiva mängden: sändare inom en inspelning, moder över minibatchen. Vid inferens klustrar DBSCAN varje gren för sig. Ingen softmax över katalogen används.

På 960 överlappningsfria testfönster återfinner vanlig självuppmärksamhet redan båda partitionerna på de flesta fönster (medelvärde ARI 0,944 för sändare, 0,986 för moder). Roterande kodning av ankomsttid, och en parvis fysisk uppmärksamhetsbias, höjer sändar-ARI till 1,000 och tar bort den kvarvarande felsvansen. Roterande kodning av plan infallriktning på avinterfolieringsgrenen hjälper mindre. Biasen når samma sändarsiffror som tidskodningen och kostar extra minne och kodartid på parvisa kartor av storlek $L \times L$. Att använda båda mekanismerna tillsammans sänker sändar-ARI till 0,853.

En Vanilla-kodare tränad bara för avinterfoliering slår den gemensamma modellen på sändare (ARI 0,999 mot 0,944), medan dess otränade modgren slår ihop kluster. En Vanilla-kodare tränad bara för moder når samma modnivå som den gemensamma modellen (ARI 0,984), medan dess otränade sändargren delar upp sändare i för många kluster. Det gemensamma målet är alltså en avvägning, inte ett gratis extra huvud. Modulationstyper utanför träningskatalogen och operativa inspelningar utvärderas inte.

Nyckelord: transformer, djup klustring, signalspaning, radarsändaridentifiering, modigenkänning, övervakad kontrastiv inlärning.

Acknowledgments

I would like to thank my supervisor at KTH, Saikat Chatterjee, and my supervisors at Saab, Alexander Karlsson and Elin Ohlman, for their guidance throughout this project. I am also grateful to Magnus Jansson for acting as examiner, and to Saab for hosting the work.

Stockholm, August 2026
Markus Johnson Swegmark

Contents

List of Figures

List of Tables

List of Acronyms

AMI adjusted mutual information

AOA angle of arrival

ARI adjusted Rand index

BERT Bidirectional Encoder Representations from Transformers

CNN convolutional neural network

DBSCAN Density-based spatial clustering of applications with noise

DEC deep embedded clustering

DL deep learning

ELINT electronic intelligence

ESM electronic support measures

FFN feed-forward network

GMM Gaussian mixture model

IDEC improved deep embedded clustering

MHA multi-head attention

ML machine learning

MLP multilayer perceptron

PDW pulse descriptor word

PRF pulse repetition frequency

PRI pulse repetition interval

RF radio frequency

RNN recurrent neural network

RoPE rotary positional encoding

TOA time of arrival

UMAP uniform manifold approximation and projection

V-measure harmonic mean of homogeneity and completeness

CHAPTER 1

Introduction

1.1 Background and Motivation

Passive ESM does not transmit. It records a time-ordered stream of PDWs and supports ELINT: which radars are present, which pulses belong to the same emitter, and which operating mode is in use [1, 2]. A transmitter that illuminates a scene is also visible to anyone who can receive it. Several emitters occupy the same band, so the listener sees a mixed stream rather than a single echo.

Modern radars vary carrier frequency, pulse width, amplitude, and PRI from burst to burst. Fingerprints built on a stable pulse train then weaken, and parsers written for regular trains become less reliable. The same emitter also switches mode: a transmit schedule chosen for a task such as volume search or track. Knowing the mode constrains how the waveform may evolve, and it can separate emitters that look similar at the type level.

When several agile emitters occupy the same window, those parsers fail. The labels available in training data are asymmetric: emitter identifiers are unique only within a recording, while operating modes come from a catalogue shared across recordings.

This thesis asks whether a transformer encoder, trained with clustering objectives, can map such streams to embeddings that cluster by emitter and by mode.

1.2 Problem Statement

Passive ESM produces a time-ordered stream of PDWs in which pulses from several emitters are interleaved and each emitter may change mode over time (Section 1.1). Operators need both a grouping by emitter and a grouping by mode from the same stream. Classical pipelines usually deinterleave first and then apply separate mode heuristics or classifiers. That split does not produce embeddings that carry both partitions, and it scales poorly when waveforms are agile and trains overlap.

This thesis treats the two partitions as a joint learning problem. Training data come from a scenario simulator (Section 4.2) in which every pulse has two labels: an emitter identity that is unique only inside its recording, and a mode drawn from a catalogue shared across recordings. The encoder maps each window of pulses to two embedding sequences. A clustering step on each sequence recovers the partitions, without assigning names from a fixed catalogue. Emitter clustering is recording-local and does not assume a known number of emitters. Mode labels are used in training to pull same-mode pulses together; at inference, mode clusters are also formed per window. The experiments

measure agreement with the simulator partitions on mixture windows of fixed length.

1.3 Research Questions

The thesis addresses the following questions:

RQ1 Given a bounded window of PDWs, can a transformer encoder deinterleave pulses and recover operating modes by clustering?

RQ2 Does a joint objective that couples deinterleaving with mode clustering outperform a comparable transformer trained for deinterleaving alone?

RQ3 Do a pairwise physical attention bias and rotary positional encoding on pulse coordinates improve clustering relative to standard multi-head attention?

1.4 Objectives and Scope

1.4.1 Objectives

We implement a transformer encoder that maps windows of PDWs to emitter and mode embeddings, cluster each embedding independently, and measure agreement with simulator labels using AMI, ARI, and V-measure (Sections 4.1 and 4.3 and Chapter 5). We train with two supervised contrastive terms, recording-local emitter positives and batch-wide mode positives, and compare that joint loss with a deinterleaving-only baseline of similar capacity (Section 4.5 and Chapter 5). We compare standard self-attention with a physical pairwise bias and with rotary positional encoding on pulse coordinates (Section 4.4 and Chapter 5).

1.4.2 Delimitations

The model sees PDWs, not raw radio-frequency samples. Pulse detection, receiver processing, and multi-platform fusion are out of scope.

All data come from a proprietary scenario simulator (Section 4.2). No operational ELINT recordings are used, so the results do not speak to real-world distribution shift. Dropped pulses and spurious detections, when present, are simulator artifacts.

The study is offline analysis of recorded streams, not real-time inference on embedded hardware. The encoder does not name emitters against a classified library. Self-attention scales with the square of window length, so each recording is split into windows of fixed length (Section 4.2.2). Scores therefore describe clustering on those windows, not tracking over an unbounded stream.

Every evaluation scenario uses the same mode catalogue as training (Section 5.2). Held-out modulation types are not tested.

1.5 Contributions

The main contributions of this thesis are:

- We propose a transformer encoder with a shared trunk and two branches that map a window of PDWs to emitter and mode embeddings, decoded by per-window DBSCAN (Section 4.3).
- We train both branches with supervised contrastive losses that differ only in the scope of the positive set: recording-local emitter identifiers versus global mode labels (Section 4.5).
- We evaluate a pairwise physical attention bias and a rotary positional encoding on time and Euclidean incidence as drop-in changes to standard self-attention (Section 4.4).
- We test the method on a labelled synthetic scenario corpus and report emitter and mode clustering with AMI, ARI, and V-measure (Chapter 5).

1.6 Ethical and Societal Considerations

Methods for passive ESM and ELINT are dual-use. The same pulse groupings that support defensive situational awareness can, in other hands, support targeting or surveillance. This thesis works on synthetic pulse features and does not treat deployment or live operational data.

The experiments use a scenario simulator rather than collections from live emitters. That avoids handling classified recordings, and the scores do not speak to operational data. If the method were later applied to such recordings, access, purpose, and retention would have to follow the relevant regulation.

Training a transformer uses more energy than a classical deinterleaving heuristic. The present runs are a small number of models of fixed size.

1.7 Thesis Outline

Chapter 2 covers pulsed radar, PDWs, emitters and modes, the ELINT chain, cluster agreement scores, and the machine-learning and transformer background used later. Chapter 3 surveys classical emitter identification, supervised radar models, deep clustering, sequence transformers, and joint clustering, and states the gap that motivates the method. Chapter 4 defines the problem, the preprocessing, the encoder and losses, the attention variants, training, the choice of scores, and baselines. Chapter 5 reports setup, the synthetic corpus, hyperparameters, joint Vanilla scores, objective ablations, attention variants, and qualitative plots. Chapter 6 interprets the results, compares

them with prior work, and states limitations. Chapter 7 answers the research questions and lists future work.

CHAPTER 2

Background

2.1 Radar Systems and Passive Reception

Pulsed radar

A radar transmits a short electromagnetic pulse, waits for the echo that scatters from objects in the environment, and infers range and motion from that return [2].

Distance follows from the round-trip delay τ between transmission and reception. With wave speed c, the range of a monostatic radar (transmitter and receiver on the same platform and thus approximately the same position) is

$$R = \frac{c\tau}{2}, \tag{2.1}$$

where the factor of 2 accounts for the outbound and return paths [2].

Radars emit pulses at a PRI, or equivalently a PRF. If an echo from an earlier pulse arrives after the next pulse has been sent, the measured delay no longer identifies a unique range.

Active radar versus passive reception

An active radar times the echo against its own outgoing pulses and from that delay obtains range [2]. The same pulses are visible to any receiver in range, so transmission also discloses presence, location, and activity [1].

Passive systems such as ESM and ELINT do not transmit. They listen to other radars to determine which emitters are present and how those emitters are behaving [1]. Without control of the waveform, the receiver is limited to the quantities it can measure after detection. Those measurements are stored as the PDWs defined next.

2.1.1 Pulse descriptor words

A receiver records a voltage time series at the antenna. Pulses appear as brief rises in energy above the noise floor. For each detection the receiver estimates carrier frequency, pulse width, time of arrival, amplitude, and, when directional antennas are available, angle of arrival.

Those estimates are stored as a PDW:

$$\boldsymbol{x}_n = \left(f_n,\, \mathrm{PW}_n,\, t_n,\, A_n,\, \theta_n,\, \ldots\right)^{\top} \in \mathbb{R}^d, \tag{2.2}$$

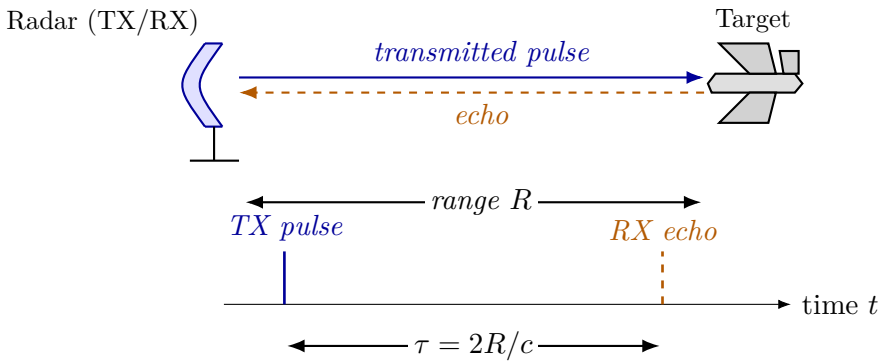


Figure 2.1. Monostatic radar geometry and round-trip timing. A co-located transmit/receive (TX/RX) aperture on the left emits a pulse that travels to a target at range R, scatters, and returns to the same aperture as an echo. The lower timeline shows the corresponding events: the outgoing TX pulse at $t = 0$ and the received echo at $t = \tau$, separated by the round-trip delay $\tau = 2R/c$ from Equation (2.1).

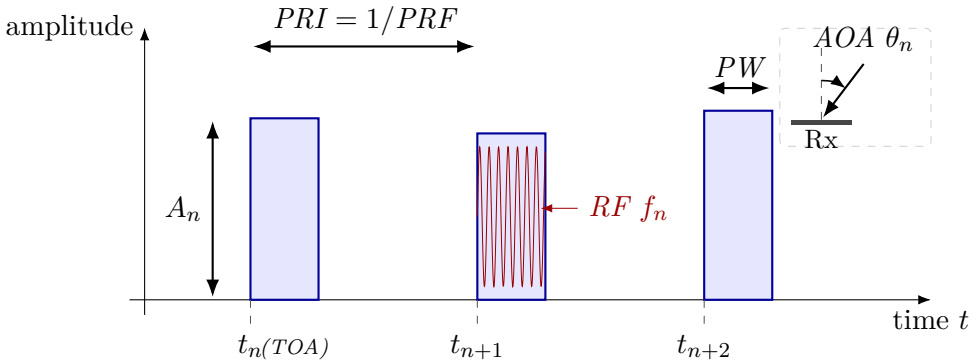


Figure 2.2. Parameters captured by a PDW for a short pulse train. Each pulse contributes a TOA (the leading edge t_n on the time axis), a pulse width (PW), a peak amplitude A_n, and a carrier RF f_n, shown as the fast oscillation inside the middle pulse. Successive pulses also define a PRI (equivalently a PRF $= 1/\text{PRI}$). The inset shows the AOA θ_n measured at the receive aperture relative to a boresight reference. A typical PDW stacks these per-pulse measurements into a vector $\boldsymbol{x}_n = (f_n, \text{PW}_n, t_n, A_n, \theta_n)$ [1].

where f_n is the measured carrier frequency, PW_n is pulse width, t_n is time of arrival (often the leading edge), A_n is amplitude, and θ_n is angle of arrival if available. Further entries, such as elevation, may be included, but once the template is chosen every pulse is stored in the same format. Figure 2.2 shows this five-parameter template.

A sequence of pulses from one transmitter is a *pulse train*. When that transmitter remains in one mode, the inter-arrival times

$$\tau_n = t_n - t_{n-1}, \qquad n \geq 2 \tag{2.3}$$

are regular or follow a programmed schedule. That interval is the PRI of the train, and its inverse is the PRF. Simple emitters keep a fixed interval. Agile emitters vary it in a planned way.

Each measured descriptor is a noisy, possibly incomplete observation of the true

pulse. Weak pulses may be missed, and noise events may be reported as pulses.

After this step the sampled voltage is discarded. The input to later stages is a time-ordered sequence of PDWs $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$, called a *recording* because it generally interleaves the pulse trains of several transmitters.

2.1.2 Emitters and operating modes

In passive ESM and ELINT, an *emitter* is a physical transmitter: pulses that share timing, frequency, and direction closely enough to be treated as one source [1]. An *operating mode* is the transmit schedule in force at a given time: carrier plan, pulse width, repetition pattern, and scan or switching rules, chosen for a task such as search, acquisition, track, or illumination [1, 2].

These are two different groupings of the same pulses. One emitter switches among modes as its task changes, so a single physical source produces several mode segments over a mission. Different emitters can also run the same functional mode with different numerical parameters, for example two trackers with distinct nominal PRFs [1].

The classical chain therefore has to answer both: which transmitter is present, and what that transmitter is doing now. Section 2.2 describes how it does so in sequence.

2.2 The Classical ELINT Pipeline

A passive collector produces a time-ordered list of PDWs and then has to answer two questions about that list: which physical emitter sent each pulse, and which operating mode was in use. Classical ELINT and ESM architectures answer those questions in stages rather than jointly [1]. Implementations differ by band and platform, but the order of work is the same: detect and measure, deinterleave into pulse trains, then analyze mode on each train.

Detection and pulse measurement

Antennas and RF conditioning capture a band of interest. A detector marks candidate pulses against noise and clutter. Parameter estimation then attaches the per-pulse measurements of Section 2.1.1, producing the PDW list that later stages consume. Estimates are noisy, weak pulses may be missed, and false alarms may enter the list with no associated radar.

The list is still mixed. Pulses from several emitters, possibly in several modes, share one time-ordered recording.

2.2.1 Deinterleaving: grouping pulses by emitter

Deinterleaving, also called sorting, assigns each pulse to a physical emitter. Its output is a set of pulse trains, one hypothesized sequence per emitter [1]. This recovers the

emitter grouping of Section 2.1.2, not a name from a catalogue.

When trains are sparse and nearly periodic, the PRI inferred from TOA differences is a usable timing signature. Histogram methods such as CDIF and SDIF accumulate inter-pulse intervals and treat peaks as candidate PRIs [1, 3]. They fail when timing is agile, and when missing pulses break sequential differencing. Overlapping trains are a further problem, because differences across emitters create false peaks.

When timing is not enough, pulses are clustered in a joint space of RF, pulse width, AOA, and related measurements, using k-means or density-based methods [3, 4]. Deinterleaving is then a clustering problem in a chosen feature space. Cluster count, metric, and scaling have to be set by hand, which is awkward when the number of emitters is unknown.

Classical architectures treat train formation as an early, discrete step. Later stages are expected to see one dominant hypothesis per emitter, or a short ranked list, rather than the interleaved mixture [1].

2.2.2 Mode analysis: grouping pulses by operating mode

Mode analysis asks a different question of the same pulses: which transmit schedule is active. In the classical chain this step runs after deinterleaving, on each recovered train rather than on the mixture [1]. The train is matched to a mode library of nominal RF bands, PRI/PRF families, stagger laws, and scan programs.

A correctly deinterleaved train can still contain several mode segments, because one emitter switches modes as its task changes. A mode match is not an emitter identity either, because several emitters can implement the same functional mode. When the library is incomplete, analysts group trains, or segments of trains, by similar behavior and name those groups later.

Equipment identification is a further lookup. Aggregated train statistics are compared with an emitter library to propose a radar type. That step names hardware against a catalogue. It is not required in order to form the emitter and mode partitions, and it is not the subject of this thesis.

Errors in deinterleaving enter mode analysis as corrupted trains. Nothing in this chain produces both groupings from the mixed stream at once.

2.3 Machine Learning Fundamentals

ML replaces hand-coded decision rules with parametric models fitted to data. A model is a function $f_{\boldsymbol{\theta}} \colon \mathcal{X} \to \mathcal{Y}$ from an input space $\mathcal{X}$ to a task-specific output space $\mathcal{Y}$. Learning searches over parameters $\boldsymbol{\theta}$ so that $f_{\boldsymbol{\theta}}$ matches the training examples under a chosen loss.

The usual training objective is empirical risk minimization. Given a dataset $\mathcal{D} =$

$\{(\boldsymbol{x}_i, y_i)\}_{i=1}^N$ and a pointwise loss ℓ, the learner solves

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \ell\Big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i),\, y_i\Big) + \Omega(\boldsymbol{\theta}), \tag{2.4}$$

where Ω is an optional regularizer. Generalization is measured on held-out data.

In supervised learning each $\boldsymbol{x}_i$ has a label y_i, and ℓ penalizes disagreement with that label. In unsupervised learning there are no labels, and the aim is to expose structure, for example by clustering. This thesis is supervised at training time: recordings carry per-pulse emitter and mode labels (Sections 4.1 and 4.2). At inference the labels are recovered by clustering embeddings, so the clustering tools of Section 2.4 and the contrastive losses of Section 2.6 still matter. Section 2.7 describes the encoder.

2.4 Clustering Methods

Clustering partitions a finite set of points $\{\boldsymbol{z}_i\}_{i=1}^N \subset \mathbb{R}^d$ into groups so that points inside a group are more similar to one another than to points in other groups. In this thesis the points are pulse embeddings and the groups are emitters within a window or operating modes. The clustering step runs only at inference, after the encoder has been trained (Section 4.3). Classical algorithms differ in how similarity is defined, whether the number of clusters must be known in advance, and how they treat outliers. Centroid methods such as k-means, and mixture models such as a GMM, require a prescribed number of groups and typically assign every point. When that number is unknown and some points should remain unassigned, density-based clustering is a better match.

DBSCAN

DBSCAN recovers clusters from local density rather than from a fixed set of centres [4]. Two hyperparameters define the density criterion: a neighbourhood radius $\varepsilon > 0$ and a minimum neighbourhood size minPts $\in \mathbb{N}$. A point is a core point if at least minPts points (including itself) lie inside its Euclidean ball of radius ε. Points that are not cores but fall inside some core's ball are border points. The remainder are labelled noise. Clusters grow by connecting core points that fall in each other's ε-neighbourhoods and attaching their border points. The number of clusters is then an output of the run, not an input.

This matches the inference setting of Equation (4.14): within each window the number of active emitters (and, separately, of modes) is not known in advance, and sparse outliers can be left unassigned instead of distorting the partition. The main practical cost is sensitivity to ε. A single global radius assumes that all clusters share a comparable density scale. The method chapter uses DBSCAN with frozen $(\varepsilon, \text{minPts})$ on each branch embedding.

Applied to raw high-dimensional features, density thresholds degrade because neighbourhoods become less informative as d grows. Deep clustering therefore first learns a lower-dimensional embedding in which the desired partitions are easier to separate, then runs a classical operator such as k-means or DBSCAN in that space [5]. How a recovered partition is scored against a reference is defined in Section 2.5. Section 2.6 reviews the representation-learning side of that pipeline.

2.5 Cluster Agreement Metrics

A clustering algorithm returns a labelling of N points. The integers used as cluster names are arbitrary: two labellings that group the same points together are the same partition. External evaluation therefore compares a predicted partition $\hat{Y}$ with a reference partition Y in a way that does not depend on those names.

All scores below are computed from the contingency table

$$n_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i,\, \hat{y}_n = j], \tag{2.5}$$

with row sums $a_i = \sum_j n_{ij}$ and column sums $b_j = \sum_i n_{ij}$ equal to the sizes of the reference and predicted clusters.

Adjusted Rand index

Each unordered pair of points is either together or apart in a given partition. A cluster of size a_i contributes $\binom{a_i}{2}$ co-clustered pairs, so

$$S_{Y\hat{Y}} = \sum_{i,j} \binom{n_{ij}}{2}, \qquad S_Y = \sum_i \binom{a_i}{2}, \qquad S_{\hat{Y}} = \sum_j \binom{b_j}{2} \tag{2.6}$$

count the pairs that share a cluster in both labellings, in Y, and in $\hat{Y}$, out of $\binom{N}{2}$ pairs in total.

The Rand index is the fraction of pairs on which Y and $\hat{Y}$ agree: together in both, or apart in both [6],

$$R(Y, \hat{Y}) = 1 - \frac{S_Y + S_{\hat{Y}} - 2\,S_{Y\hat{Y}}}{\binom{N}{2}} \in [0, 1]. \tag{2.7}$$

The numerator $S_Y + S_{\hat{Y}} - 2\,S_{Y\hat{Y}}$ is the number of pairs that are together in one partition and apart in the other.

When most points lie in different clusters, many pairs are apart in both labellings by chance, so R stays close to one even when the partitions disagree.

If the predicted labels are shuffled at random, keeping the cluster sizes, any fixed pair is together in $\hat{Y}$ with probability $p = S_{\hat{Y}}/\binom{N}{2}$, which lies in $[0, 1]$. Only the S_Y

pairs that are already together in Y can contribute to $S_{Y\hat{Y}}$. Each of them remains together in $\hat{Y}$ with probability p, so the expected overlap is that count times p,

$$\mathbb{E}[S_{Y\hat{Y}}] = S_Y \cdot \frac{S_{\hat{Y}}}{\binom{N}{2}}. \tag{2.8}$$

With the sizes fixed, R in Equation (2.7) moves only with $S_{Y\hat{Y}}$. Subtracting this chance overlap, and dividing by the overlap at a perfect match, $\frac{1}{2}(S_Y + S_{\hat{Y}})$, gives [6]

$$\mathrm{ARI} = \frac{S_{Y\hat{Y}} - \mathbb{E}[S_{Y\hat{Y}}]}{\frac{1}{2}\Big(S_Y + S_{\hat{Y}}\Big) - \mathbb{E}[S_{Y\hat{Y}}]}, \qquad \mathrm{ARI} \le 1. \tag{2.9}$$

Then $\mathrm{ARI} = 1$ at exact recovery up to relabelling, $\mathrm{ARI} \approx 0$ at chance, and negative values are worse than random.

Adjusted mutual information

Mutual information is the reduction in uncertainty about one labelling given the other [7],

$$\mathrm{MI}(Y;\hat{Y}) = H(Y) - H(Y \mid \hat{Y}) = H(\hat{Y}) - H(\hat{Y} \mid Y). \tag{2.10}$$

$H(Y \mid \hat{Y})$ is the uncertainty left in Y once $\hat{Y}$ is known, and likewise with the labels swapped. The quantity is zero when the labellings are independent, and equals $H(Y)$ or $H(\hat{Y})$ when one labelling determines the other.

The Shannon entropies of the cluster-size distributions are

$$H(Y) = -\sum_{i:\, a_i>0} \frac{a_i}{N} \log \frac{a_i}{N}, \qquad H(\hat{Y}) = -\sum_{j:\, b_j>0} \frac{b_j}{N} \log \frac{b_j}{N}. \tag{2.11}$$

The plug-in estimate of Equation (2.10) from Equation (2.5) is

$$\mathrm{MI}(Y;\hat{Y}) = \sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{N\, n_{ij}}{a_i b_j}, \tag{2.12}$$

with the convention $0 \log 0 = 0$.

The expectation of MI under the same permutation null grows with the number of clusters. The adjusted mutual information AMI subtracts that baseline and rescales by the average of the marginal entropies [7],

$$\mathrm{AMI}(Y;\hat{Y}) = \frac{\mathrm{MI}(Y;\hat{Y}) - \mathbb{E}[\mathrm{MI}]}{\frac{1}{2}\Big(H(Y) + H(\hat{Y})\Big) - \mathbb{E}[\mathrm{MI}]}, \tag{2.13}$$

when the denominator is positive; otherwise $\mathrm{AMI} \equiv 0$. The range matches ARI. AMI weights clusters more evenly than pair counting, so large groups dominate it less.

Homogeneity, completeness, and V-measure

Homogeneity is high when each predicted cluster contains points from only one reference class. Completeness is high when each reference class is collected into a single predicted cluster. Splitting a class across many predicted clusters preserves homogeneity and lowers completeness; merging several classes into one predicted cluster does the reverse.

Expanding the conditional entropies of Equation (2.10) gives

$$H(Y \mid \hat{Y}) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{b_j}, \qquad H(\hat{Y} \mid Y) = -\sum_{i,j:\, n_{ij}>0} \frac{n_{ij}}{N} \log \frac{n_{ij}}{a_i}. \tag{2.14}$$

Homogeneity and completeness are then [8]

$$h = 1 - \frac{H(Y \mid \hat{Y})}{H(Y)}, \qquad c = 1 - \frac{H(\hat{Y} \mid Y)}{H(\hat{Y})}, \tag{2.15}$$

with $h = 1$ if $H(Y) = 0$ and $c = 1$ if $H(\hat{Y}) = 0$. The V-measure is their harmonic mean,

$$\mathrm{V}(Y, \hat{Y}) = \frac{2hc}{h + c}, \qquad h + c > 0, \tag{2.16}$$

and $\mathrm{V} = 0$ if $h = c = 0$. The harmonic mean is small if either factor is small, so a high V-measure requires both.

The three families return one at exact recovery and about zero at chance. Between those extremes, ARI follows pair agreement, AMI follows shared information, and homogeneity versus completeness separates splitting from merging.

2.6 Deep Representation Learning

Many DL pipelines replace hand-crafted features with a parametric map trained from data. The central object is an encoder $f_{\boldsymbol{\theta}} \colon \mathcal{X} \to \mathbb{R}^d$ that yields an embedding $\boldsymbol{z} = f_{\boldsymbol{\theta}}(\boldsymbol{x})$. Later steps use $\boldsymbol{z}$ rather than the raw coordinates.

When labels exist, $f_{\boldsymbol{\theta}}$ can be trained so that $\boldsymbol{z}$ separates classes. When the target is a partition, the same encoder can be trained so that same-group points lie close and different-group points lie far. Contrastive objectives do this by treating some pairs as positives and others as negatives. Let $\boldsymbol{z}_i$ and $\boldsymbol{z}_i^+$ be normalized embeddings of a positive pair, let $\mathrm{sim}(\cdot, \cdot)$ be cosine similarity, and let $\tau > 0$ be a temperature. An InfoNCE-style loss is

$$\mathcal{L}_{\mathrm{NCE}}(i) = -\log \frac{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big)}{\exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_i^+)/\tau\Big) + \sum_{j \neq i} \exp\Big(\mathrm{sim}(\boldsymbol{z}_i, \boldsymbol{z}_j^+)/\tau\Big)}, \tag{2.17}$$

which is the form used, with a larger positive set, in the supervised contrastive kernel of Section 4.5 [9, 10, 11].

Sequence encoders based on self-attention instantiate $f_{\boldsymbol{\theta}}$ when the input is an ordered

set of tokens, here a window of PDWs. Section 2.7 reviews that encoder. Section 3.3 surveys deep clustering methods that couple representation learning with a discrete partition.

2.7 The Transformer Encoder

The transformer architecture replaces recurrence with a stack of layers built around *self-attention*, so that every position in a sequence can directly attend to every other position in parallel [12]. Encoder-only stacks, as in BERT [13], map an input sequence to contextualised token representations of the same length and are a natural choice when the downstream task is representation learning or clustering over tokens rather than autoregressive generation.

Scaled dot-product attention. Attention maps three matrices of *queries* $\boldsymbol{Q}$, *keys* $\boldsymbol{K}$, and *values* $\boldsymbol{V}$ to an output matrix of the same leading dimension as $\boldsymbol{Q}$. Each row of $\boldsymbol{Q}$ scores all rows of $\boldsymbol{K}$ through inner products; the scores are scaled, normalised with a row-wise softmax, and used to form a convex combination of the rows of $\boldsymbol{V}$. With d_k the dimension of each query and key vector (column width of $\boldsymbol{Q}$ and $\boldsymbol{K}$ after the usual transpose convention), the mapping is

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\boldsymbol{V}. \tag{2.18}$$

The scaling by $\sqrt{d_k}$ keeps the inner products from growing too large in magnitude as d_k increases, which would otherwise drive the softmax into near one-hot rows and yield vanishing gradients [12]. Equation (2.18) is reused later as the core attention primitive inside the encoder.

Self-attention and MHA. In *self-attention*, $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ are all linear projections of the same input sequence layout, so a token's representation is updated by aggregating information from the full sequence. MHA runs h attention mechanisms in parallel on different learned subspaces, concatenates the head outputs, and applies one more linear map to restore the model width [12]. The heads specialise to different relational patterns while sharing the same residual and normalisation wrapper as a single conceptual operator.

Positional information. Self-attention is permutation-equivariant in its inputs: reordering the rows of $\boldsymbol{Q}$, $\boldsymbol{K}$, and $\boldsymbol{V}$ in the same way reorders the output rows accordingly, but does not otherwise encode order. Transformer encoders therefore inject a positional signal. The original architecture adds fixed sinusoidal features or learned embeddings to each token vector before the first layer [12, 13]. Those encodings are absolute. They tag a token with its index, not with its distance to other tokens.

Rotary positional encoding. Su et al. [14] introduce RoPE. The query and key of each token are rotated by a phase that depends on a scalar coordinate p, leaving the token embeddings unchanged. In the original formulation p_i is the token index i. Write $\boldsymbol{q}_i = \boldsymbol{W}_q \boldsymbol{h}_i$ and $\boldsymbol{k}_i = \boldsymbol{W}_k \boldsymbol{h}_i$ for the usual per-head projections of a token representation $\boldsymbol{h}_i$. The rotated vectors are functions $f_q(\boldsymbol{h}_i, p_i)$ and $f_k(\boldsymbol{h}_j, p_j)$ of the hidden representation and the coordinate. Their inner product is required to depend on the two coordinates only through their difference,

$$\Big\langle f_q(\boldsymbol{h}_i, p_i),\, f_k(\boldsymbol{h}_j, p_j) \Big\rangle = g(\boldsymbol{h}_i, \boldsymbol{h}_j, p_j - p_i). \tag{2.19}$$

Assume d_k is even. The d_k coordinates of a head are grouped into $d_k/2$ two-dimensional planes. Plane m is rotated by the angle $p\omega_m$, where $\omega_m = b^{-2(m-1)/d_k}$ and $b = 10^4$ is a fixed base. Because p increases by one between consecutive tokens, the first plane rotates by one radian per token. The base b sets the other end of the ladder: the last plane rotates by about p/b radians and completes a full turn only after p reaches $2\pi b$. Fast-rotating planes resolve small gaps in p. Slow-rotating planes resolve large ones. The same geometric progression of wavelengths appears in the sinusoidal encoding of Vaswani et al. [12]. Each plane uses the rotation

$$\boldsymbol{\Omega}(\phi) = \begin{bmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{bmatrix}, \tag{2.20}$$

and the head-wide map is the block-diagonal matrix

$$\boldsymbol{R}(p) = \operatorname{diag}\Big(\boldsymbol{\Omega}(p\omega_1),\, \boldsymbol{\Omega}(p\omega_2),\, \ldots,\, \boldsymbol{\Omega}(p\omega_{d_k/2})\Big). \tag{2.21}$$

Applying $\boldsymbol{R}(p)$ after the projections gives

$$\tilde{\boldsymbol{q}}_i = \boldsymbol{R}(p_i)\, \boldsymbol{q}_i, \qquad \tilde{\boldsymbol{k}}_j = \boldsymbol{R}(p_j)\, \boldsymbol{k}_j. \tag{2.22}$$

Planar rotations compose by subtracting angles, $\boldsymbol{\Omega}(\phi)^\top \boldsymbol{\Omega}(\psi) = \boldsymbol{\Omega}(\psi - \phi)$. Block-wise this is $\boldsymbol{R}(p_i)^\top \boldsymbol{R}(p_j) = \boldsymbol{R}(p_j - p_i)$, so

$$\Big\langle \tilde{\boldsymbol{q}}_i,\, \tilde{\boldsymbol{k}}_j \Big\rangle = \boldsymbol{q}_i^\top\, \boldsymbol{R}(p_j - p_i)\, \boldsymbol{k}_j. \tag{2.23}$$

Each token is rotated by its own coordinate, yet the logit sees only the difference. Setting $\boldsymbol{R}(p) = \boldsymbol{I}$ recovers the content-only inner product. Figure 2.3 shows the geometry on a single plane.

The construction applies to any real scalar p, not only to integer indices. When the token features already carry timing, a separate positional encoding is sometimes omitted altogether. The method chapter returns to this for PDW windows: a baseline that omits a separate encoding, and a rotary encoding that uses physical coordinates in place of token indices.

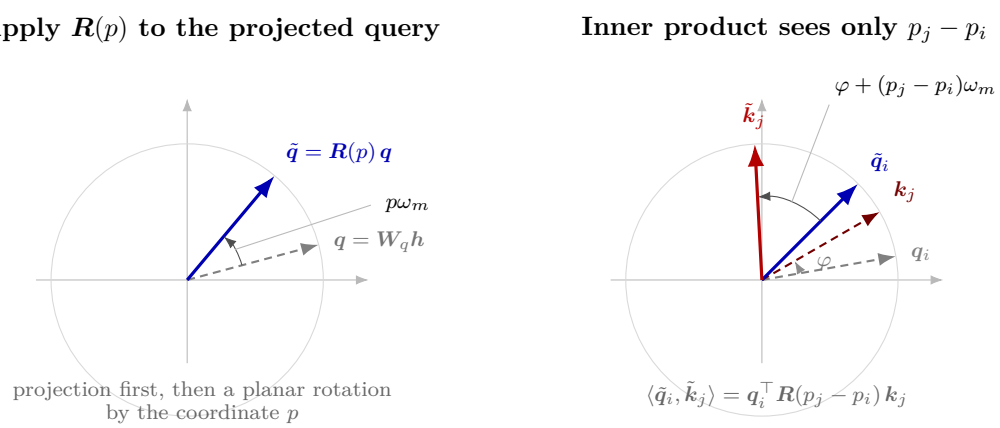


Figure 2.3. Geometry of RoPE on one two-dimensional plane of a head. Left: the projected query $\boldsymbol{q} = \boldsymbol{W}_q\boldsymbol{h}$ (dashed) is rotated by $\boldsymbol{R}(p)$ from Equation (2.21), which adds the phase $p\omega_m$. Right: the angle between the rotated query and key equals the content angle φ plus the relative phase $(p_j - p_i)\omega_m$. The inner product depends on the coordinate difference alone (Equation (2.23)).

Encoder layer. A standard transformer *encoder layer* alternates two sub-layers: MHA followed by a position-wise FFN applied independently at every position. Each sub-layer uses a residual connection and layer normalisation [12]. The FFN is typically a two-layer MLP with a wide inner dimension and a nonlinear activation, acting as the per-token nonlinearity while MHA handles cross-token mixing. Stacking N such layers yields a deep encoder that maps an input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$ to contextualised representations $(\boldsymbol{h}_1^{(N)}, \ldots, \boldsymbol{h}_L^{(N)})$. Dense self-attention scales as L^2, so the method chapter fixes a window length.

CHAPTER 3

Related Work

This chapter reviews work that the method in Chapter 4 builds on or differs from. The first two sections cover radar-specific pipelines: classical sorting and identification, then learned classifiers and embedding models for PDWs. The remaining sections cover the learning ingredients used later—deep clustering, transformers on irregular sequences, and clustering at more than one granularity. Each section ends with the limitation that the next chapter addresses.

3.1 Classical Radar Emitter Identification

Operational ELINT systems recover emitter identity from measured PDWs by combining pulse sorting with library lookup [1, 2]. After detection, pulses are grouped into trains and each train is matched against stored templates of carrier frequency, pulse width, PRI family, and scan behavior. The grouping step is deinterleaving; the matching step is identification. Both historically depend on hand-specified features and on regularity that agile radars no longer guarantee.

The oldest deinterleavers exploit repetition in TOA. The cumulative difference histogram (CDIF) accumulates TOA differences at increasing pulse lags and treats histogram peaks as candidate PRIs, which a subsequent search then validates as pulse sequences [15]. The sequential difference histogram (SDIF) keeps only the current difference level, with a derived detection threshold, which avoids accumulating every lag while remaining a TOA-only method [16]. These algorithms are reliable when each emitter keeps an approximately constant PRI and when missing pulses and cross-emitter differences do not create false peaks. They degrade under stagger, jitter, and dense mixing, which is the setting of the recordings in Section 4.1.

When timing alone is unstable, sorting moves into a joint space of RF, pulse width, and AOA. Centroid methods such as k-means and density clustering such as DBSCAN then replace histogram peak finding [3, 4]. Density clustering has the operational advantage that the number of emitters need not be specified in advance. Its accuracy still depends on the metric, the scaling of each coordinate, and a neighbourhood radius that is hard to set when cluster densities differ. Offline direction clustering of overlapping pulses is one concrete instance of this approach [17]. The method chapter uses DBSCAN only after a learned embedding; applying it to raw window features is retained as a baseline (Section 4.8).

Once trains are formed, identification is typically template matching against a mode or equipment library [1]. Statistical fingerprints—histograms of RF and PRI, dwell

statistics, scan-period estimates—summarize each train. The match assumes that the library contains the emitter and that deinterleaving errors have not corrupted the fingerprint. Novel emitters and modes that are absent from the catalogue fall outside this procedure by construction.

Classical pipelines treat deinterleaving and mode analysis as successive stages, each with its own hand-engineered features. They do not learn a representation in which both partitions are visible, and they have no mechanism for emitters that appear only at inference. Those two requirements—recording-local emitter grouping and a mode grouping that is not a closed-catalogue softmax—motivate the learned encoder of Chapter 4.

3.2 Deep Learning for Radar Signal Analysis

Learned models replace histogram rules with a parametric map from PDW sequences or time-frequency images to labels. Most published systems are supervised classifiers: they assume a global catalogue of emitter types, or of intra-pulse modulations, and they train a CNN or RNN with cross-entropy [3]. Attribute-specific recurrent networks on PDW streams are a representative sequence model in this family [18]. They capture temporal dependence that a bag of pulse parameters misses, but the output is still a class index in a fixed label set.

A closer relative is supervised contrastive training for emitter classification. Jonsson [19] trains an encoder so that samples with the same emitter label lie close in embedding space, using synthetic scenarios from the same simulator lineage as Section 4.2. The loss belongs to the family used in Section 4.5, but the task is still closed-set identification: emitter identifiers are treated as globally comparable classes, and inference is classification rather than clustering. Recurrent ensembles for emitter classification under dataset drift make the same closed-set assumption and treat out-of-distribution detection as a separate module [20].

Deinterleaving as clustering, with an unknown number of emitters, is a different problem. Gunn et al. [21] train a transformer encoder with a triplet loss so that, within one interleaved stream, pulses from the same emitter are close and pulses from different emitters are far. At inference they cluster the embeddings with hierarchical density clustering (HDBSCAN) [22, 23]. They do not assume a known emitter count, they evaluate with AMI, ARI, and V-measure, and they omit index-based positional encodings because TOA already orders the pulses. That pipeline is the closest published counterpart to Section 4.3.

Three differences remain. First, Gunn et al. produce a single embedding and a single partition—emitter identity—and treat operating mode as a later analysis that becomes easier after deinterleaving. Second, they train with a triplet hinge, which uses one positive and one negative per anchor; the method in Section 4.5 uses the supervised contrastive kernel of Khosla et al. [11], which averages over all positives of an anchor

in the mini-batch. Third, their transformer uses vanilla dot-product attention. This thesis additionally evaluates a pairwise physical bias and a rotary encoding of time and Euclidean incidence (Section 4.4).

Deep models for radar pulses already exist. What they still omit is the joint problem of this thesis: two embeddings whose positive sets have different scopes—recording-local emitters and globally shared modes—both decoded by clustering rather than by a catalogue softmax.

3.3 Deep Clustering and Contrastive Representation Learning

Classical clustering assumes that either the raw features or a fixed embedding already reveal the desired partition. Deep clustering drops that split: the encoder and the discrete structure are learned together.

DEC places trainable centroids in embedding space and alternates soft assignment with a Kullback–Leibler objective that sharpens confident assignments toward an auxiliary target [5]. IDEC adds a reconstruction term so that the clustering loss does not destroy local geometry [24]. DeepCluster instead runs k-means on the current features and treats the assignments as pseudo-labels for a discriminative update [25]. In all three, the number of clusters is a design choice, and the only partition being learned is a single flat grouping.

Contrastive methods replace explicit centroids with pairwise attraction and repulsion. InfoNCE and SimCLR pull two views of the same instance together and push other batch members apart [9, 10]. SwAV predicts a small codebook of prototypes from one view and uses the assignment of the other view as the target, which avoids comparing every pair while still preventing collapse [26]. Contrastive clustering applies the same idea in both the instance dimension and the cluster dimension of a feature matrix [27]. These objectives are unsupervised: positives are defined by augmentation, not by labels.

When labels are available, the positive set can be defined by class identity rather than by instance identity. Supervised contrastive learning does exactly that: every other sample with the same class is a positive, and the loss is an InfoNCE average over those positives [11]. It recovers the triplet loss of FaceNet as a special case that uses a single positive and a hard negative [28]. The method of Section 4.5 uses this kernel twice, with different rules for who counts as a positive.

Two properties of this literature matter for the method. First, training and assignment are usually coupled: DEC, DeepCluster, and SwAV produce cluster indices as part of the loss. This thesis decouples them. The contrastive terms shape the embedding; DBSCAN is applied only at inference and contributes no gradient (Equation (4.14)). Second, almost every method learns one partition. A single contrastive term cannot encode both a recording-local emitter grouping and a globally comparable mode group-

ing, because those two relations contradict each other in one embedding: two pulses from different emitters can share a mode, and two pulses from one emitter can differ in mode. That conflict is why the encoder splits into two branches after a shared trunk.

3.4 Transformers for Time Series and Sets

The transformer encoder maps a sequence of tokens to contextualised representations by unmasked self-attention [12, 13]. Every pulse in a window can attend to every other pulse, which matches the structure of interleaved PDWs: the next pulse of the same emitter is generally not the next token. The cost is quadratic in the window length, which is why Section 4.2.2 processes recordings in fixed-length windows.

Self-attention is permutation-equivariant. Without a positional signal it treats the window as a set, which is the setting studied by the Set Transformer [29]. Absolute sinusoidal encodings and learned position embeddings restore order for language, where the index of a token is a meaningful coordinate [12]. For PDWs that coordinate is the wrong one. Consecutive pulses are not equally spaced in time, so an ordinal position misrepresents TOA gaps. Multivariate time-series transformers typically assume regular sampling and add a time-step index in the same way [30]. Gunn et al. [21] therefore omit positional encodings and rely on TOA as an input feature. The baseline encoder of Section 4.3 makes the same choice.

Relative encodings offer a middle path. Shaw et al. [31] add a learned embedding of the discrete offset between token indices to the attention logits. Graphormer generalises that idea to pairwise biases derived from graph distances [32]. The physical attention bias in Section 4.4.1 is the same mechanism with continuous pulse parameters: pairwise absolute differences of window-local TOA and of the two planar Euclidean incidence components, each scaled by a learned per-head coefficient.

Su et al. [14] rotate the queries and keys (Section 2.7). The inner product then depends on content and on the coordinate difference. Vision models split the rotation planes across spatial axes [33]. The method replaces token indices with physical coordinates, reuses that split for the two Euclidean incidence components on the deinterleaving branch, and rotates the trunk and mode branch by TOA only (Section 4.4.2). It does not rotate by a scalar azimuth angle.

Existing positional schemes were designed for discrete indices, regular grids, or a single shared coordinate. A PDW window is an irregular point set with at least two physically distinct relations—time of arrival and arrival direction—and the two estimation tasks should not weight those relations equally. Whether injecting them as a logit bias, as a rotary phase, or not at all improves clustering is taken up in Chapter 5.

3.5 Joint and Multi-Task Clustering

When two tasks share structure, a common pattern is a shared trunk with task-specific heads, trained by a weighted sum of losses [34]. Image models often attach two softmax heads to one backbone, for example object class and attribute. That layout assumes both label sets are globally consistent catalogues. It does not by itself produce clusterable embeddings, and it cannot represent emitter identifiers that are comparable only within a recording.

Hierarchical clustering learns several partitions, but they are nested levels of one relation, not two different relations. Yang et al. [35] alternate agglomerative merges with representation updates under a triplet loss, so coarser groups are formed from finer ones of the same kind. A hierarchy of that form would be appropriate if modes were subtypes of emitters, or emitters subtypes of modes. They are not. One emitter uses several modes, and one mode appears on several emitters (Section 4.1). The two partitions also do not share a label scope: emitter symbols are recording-local, mode symbols are global.

A method that treats the two as softmax heads on a shared backbone therefore asks the wrong questions at inference. A method that learns a single embedding must compromise between two incompatible similarity relations: pulses that should be close as modes may need to be far as emitters, and conversely. The dual-branch encoder of Section 4.3 is the direct response to that conflict. The two supervised contrastive terms of Section 4.5 differ only in the scope of the positive set, and each branch is decoded by clustering rather than by a closed-catalogue classifier.

Chapter 4 therefore uses a transformer encoder on interleaved PDW streams, two contrastive terms with recording-local versus global positives, and independent density clustering of each branch at inference.

CHAPTER 4

Method

4.1 Problem Formulation

Observed input

The data consist of S recordings indexed by $s \in \{1, \ldots, S\}$, each an intercepted stream in which pulses from several emitters are interleaved in time. Recording s is observed as a finite, time-ordered sequence of PDWs

$$\boldsymbol{X}^{(s)} = (\boldsymbol{x}_1^{(s)}, \ldots, \boldsymbol{x}_{N_s}^{(s)}), \qquad \boldsymbol{x}_n^{(s)} \in \mathbb{R}^d,$$

where each $\boldsymbol{x}_n^{(s)}$ contains the measured parameters of one pulse (carrier frequency, amplitude, pulse width, time of arrival, and angles of arrival), and the length N_s varies between recordings. The encoder operates on fixed-length windows extracted from these recordings as described in Section 4.2.2. When a single recording is considered, the superscript is omitted.

Labels

Each pulse $n \in \{1, \ldots, N_s\}$ carries an emitter identity $e_n^{(s)} \in \{1, \ldots, K^{(s)}\}$ and an operating mode $m_n^{(s)} \in \{1, \ldots, M\}$. The emitter count $K^{(s)}$ may vary between recordings, and each emitter identifier is meaningful only within its recording. Mode labels, by contrast, are drawn from a finite catalogue of size M shared across all recordings. The value of M for the corpus used here is given in Section 5.2. A recording generally contains only $M^{(s)} \leq M$ unique modes. The two labels describe different relations: one emitter may switch between several modes, and the same mode may occur on several emitters. The two labels of a pulse are collected into the ground-truth label pair

$$\boldsymbol{y}_n^{(s)} = \left(e_n^{(s)},\, m_n^{(s)}\right) \in \{1, \ldots, K^{(s)}\} \times \{1, \ldots, M\}, \tag{4.1}$$

which constitutes the complete per-pulse annotation of recording s.

For recording s, the coordinates of $\boldsymbol{y}_n^{(s)}$ induce emitter and mode cells on the pulse-index set,

$$\mathcal{E}_k^{(s)} = \{\, n : e_n^{(s)} = k \,\}, \qquad k \in \{1, \ldots, K^{(s)}\}, \tag{4.2}$$

$$\mathcal{M}_\ell^{(s)} = \{\, n : m_n^{(s)} = \ell \,\}, \qquad \ell \in \{1, \ldots, M\}. \tag{4.3}$$

The emitter cells are non-empty and partition $\{1, \ldots, N_s\}$. The mode cells are also

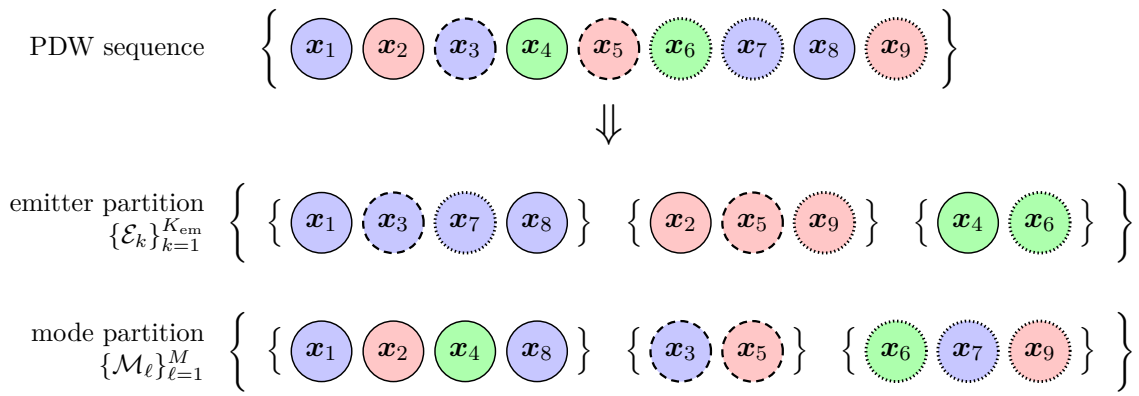


Figure 4.1. Each pulse $\boldsymbol{x}_n$ is assigned both an emitter label e_n (color) and a mode label m_n (border style).

disjoint, but $\mathcal{M}_\ell^{(s)}$ is empty when mode ℓ is absent from the recording; the non-empty mode cells form the corresponding partition. Recovering the emitter partition is the deinterleaving task, whereas recovering the mode partition groups pulses by the active mode for each PDW.

Estimation objective

At inference a time-ordered pulse sequence $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ is observed and the label pairs of Equation (4.1) are unknown. The goal is to recover the emitter and mode partitions of that sequence, up to independent relabelling of the cluster indices in each coordinate. The estimated labels

$$\hat{\boldsymbol{y}}_n = \left(\hat{e}_n,\, \hat{m}_n\right) \in \{1, \ldots, \hat{K}\} \times \{1, \ldots, \hat{M}\}, \qquad n \in \{1, \ldots, N\}, \tag{4.4}$$

induce

$$\widehat{\mathcal{E}}_k = \{\, n : \hat{e}_n = k \,\}, \quad k \in \{1, \ldots, \hat{K}\}, \qquad \widehat{\mathcal{M}}_\ell = \{\, n : \hat{m}_n = \ell \,\}, \quad \ell \in \{1, \ldots, \hat{M}\}, \tag{4.5}$$

with $\hat{K}$ and $\hat{M}$ also unknown. Elementwise equality with the ground-truth symbols is not required; only the partitions matter, and they are scored with the permutation-invariant metrics of Section 4.7.

The method does not classify pulses against a fixed catalogue. It learns a map $f_{\boldsymbol{\theta}}$ that sends $\boldsymbol{X}$ to two sequences of pulse embeddings, one for emitters and one for modes. A clustering step applied to those embeddings produces Equation (4.4). All trainable weights of $f_{\boldsymbol{\theta}}$ are collected in the parameter vector $\boldsymbol{\theta}$, which is fitted by the loss of Section 4.5; the architecture is specified in Section 4.3.

4.2 Data Representation and Preprocessing

All training and evaluation data are synthetic because operational ELINT recordings are scarce and sensitive. A scenario simulator generates time-ordered PDW recordings with varying numbers of emitters and operating modes [19]. Each pulse contains the measured parameters and per-pulse labels defined in Section 4.1. The composition of the resulting corpus is reported in Section 5.2.

Preprocessing first standardizes carrier frequency, pulse width, and log-amplitude using statistics from the training split and converts the measured AOA into Euclidean direction components. Each recording is then divided into fixed-length windows. Absolute TOA is finally min–max mapped inside each window, so that the timing feature seen by the encoder depends only on pulses that co-occur in the same attention context, thereby reducing the risk of overfitting to scenario-specific timing characteristics and providing a more realistic representation of the information available at inference time. The encoder input is therefore the $d = 7$-dimensional vector of scaled carrier frequency, pulse width, log-amplitude, window-local TOA, and the three incidence-direction components.

No hand-crafted timing features are appended to this vector. Preliminary experiments augmented each pulse with lagged timing features, namely its instantaneous ΔTOA together with rolling means and standard deviations of ΔTOA over several look-back widths, to expose the local PRI structure explicitly; this configuration consistently degraded both emitter and mode clustering relative to the plain PDW input. A plausible explanation is that such look-back statistics summarize only the pulses preceding each position and mix contributions from all interleaved emitters, so under the pulse-count imbalance of the corpus (Section 5.2) they are dominated by the densest emitters and carry little signal for rare ones, whereas unmasked self-attention (Section 4.3) can relate each pulse to every other pulse in the window and recover inter-arrival structure per emitter. Letting the encoder learn timing structure implicitly also avoids committing to a fixed set of summary statistics, which keeps the representation free to adapt to interleaving patterns and PRI schedules that differ from those seen during training.

4.2.1 Per-feature scaling

Standardization statistics are estimated across all recordings in the training split and applied unchanged to validation and test data. Carrier frequency, pulse width, and log-amplitude are standardized per feature as

$$\tilde{g}_n = \frac{g_n - \mu_g}{\sigma_g}. \tag{4.6}$$

Here μ_g and σ_g are the corresponding training-set mean and standard deviation, and amplitude is log-transformed to reduce its dynamic range. The measured AOA consists of an azimuth θ_n^{az} and a latitude θ_n^{lat}, which together specify the incidence direction of the pulse on the unit sphere. Rather than feeding these angles to the encoder directly,

each pair is converted to the Euclidean unit direction vector

$$\boldsymbol{u}_n = \left(u_n^x,\, u_n^y,\, u_n^z\right)^\top = \left(\cos\theta_n^{\mathrm{lat}} \cos\theta_n^{\mathrm{az}},\, \cos\theta_n^{\mathrm{lat}} \sin\theta_n^{\mathrm{az}},\, \sin\theta_n^{\mathrm{lat}}\right)^\top, \tag{4.7}$$

whose components lie in $[-1, 1]$ and are mapped to $[0, 1]$ by the fixed affine rescaling

$$\tilde{u}_n^c = \frac{u_n^c + 1}{2}, \qquad c \in \{x, y, z\}. \tag{4.8}$$

Neither map depends on the training corpus.

The Cartesian representation avoids the wrap-around discontinuity of a rescaled angle, which places azimuths near 0 and 2π at opposite ends of the feature range although they describe nearly the same direction. Feeding incidence as continuous direction components therefore frees the encoder from having to learn that symmetry from the input features alone.

Corpus-wide standardization of frequency, pulse width, and amplitude preserves absolute ranges that may distinguish operating modes across recordings. Absolute TOA is left in physical units at this stage and is min–max mapped only after windowing (Equation (4.10)).

4.2.2 Windowing

Standard self-attention has time and memory complexity $\mathcal{O}(L^2)$ for a window of L pulses. Because a complete recording may contain millions of pulses, the encoder processes fixed-length windows rather than entire recordings. The value of L, and the training stride, are reported in Table 5.2.

Given a recording $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)$ and stride δ, window w is

$$\boldsymbol{X}^{(w)} = \left(\boldsymbol{x}_{1+(w-1)\delta}, \ldots, \boldsymbol{x}_{L+(w-1)\delta}\right), \qquad w \in \left\{1, \ldots, \left\lfloor \frac{N-L}{\delta} \right\rfloor + 1\right\}. \tag{4.9}$$

The superscript (w) indexes windows within one recording, whereas (s) indexes recordings and is suppressed here. The number of windows therefore varies with the recording length. Any trailing segment shorter than L is dropped.

During training, choosing $\delta < L$ produces overlapping windows and therefore more training samples. A smaller stride improves coverage of sparse emitters and modes and reduces sensitivity to window boundaries, but it also repeats more of the same PDWs, increasing computation and correlation between samples. Advancing the window by a small stride resembles streaming inference because outputs can be refreshed as new pulses arrive, although it is not online learning because the model parameters remain fixed.

Reported validation and test scores use $\delta = L$, yielding non-overlapping windows that are processed entirely offline. The evaluation dump itself may be generated with the training stride and then subsampled; that protocol is stated in Section 5.4.1. Streaming

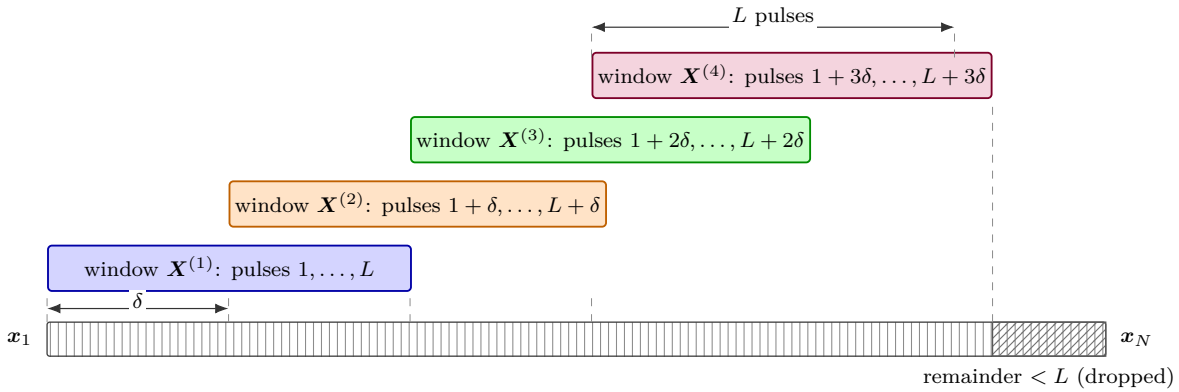


Figure 4.2. Sliding-window segmentation of a recording. Successive windows of length L advance by stride δ and overlap by $L-\delta$ pulses. A trailing segment shorter than L is dropped.

inference and online model adaptation are left to future work (Section 7.3).

Each window retains the emitter label, mode label, and recording identifier associated with every pulse. The emitter and mode labels supervise their respective branches and provide evaluation targets. This makes the model and also evaluation dependent on the selected window length L, why a fixed L is used in this thesis.

The recording identifier is not an encoder input or prediction target; it is used to keep recording-local emitter identifiers distinct and to form scenario-aware mini-batches.

After windowing, absolute TOA is min–max mapped to $[0, 1]$ using only the L pulses inside the current window:

$$\tilde{t}_i^{(w)} = \frac{t_i^{(w)} - t_{\min}^{(w)}}{t_{\max}^{(w)} - t_{\min}^{(w)}}, \tag{4.10}$$

where $t_{\min}^{(w)}$ and $t_{\max}^{(w)}$ are the minimum and maximum TOA among the pulses of window w, and i indexes positions within that window. Per-window mapping keeps the TOA channel self-contained; when training uses $\delta < L$, the same physical pulse may receive different normalized values in overlapping windows, which is the intended behaviour for a window-local feature.

4.3 Transformer Encoder Architecture

The encoder $f_{\boldsymbol{\theta}}$ takes the form of a transformer-encoder backbone [12] with a shared trunk followed by two task-specific branches for mode clustering and deinterleaving. The overall layout is shown in Figure 4.3. The trunk produces a single contextualised representation of the input window, which the branches specialise into emitter-oriented and mode-oriented embeddings for the two estimation tasks defined in Section 4.1.

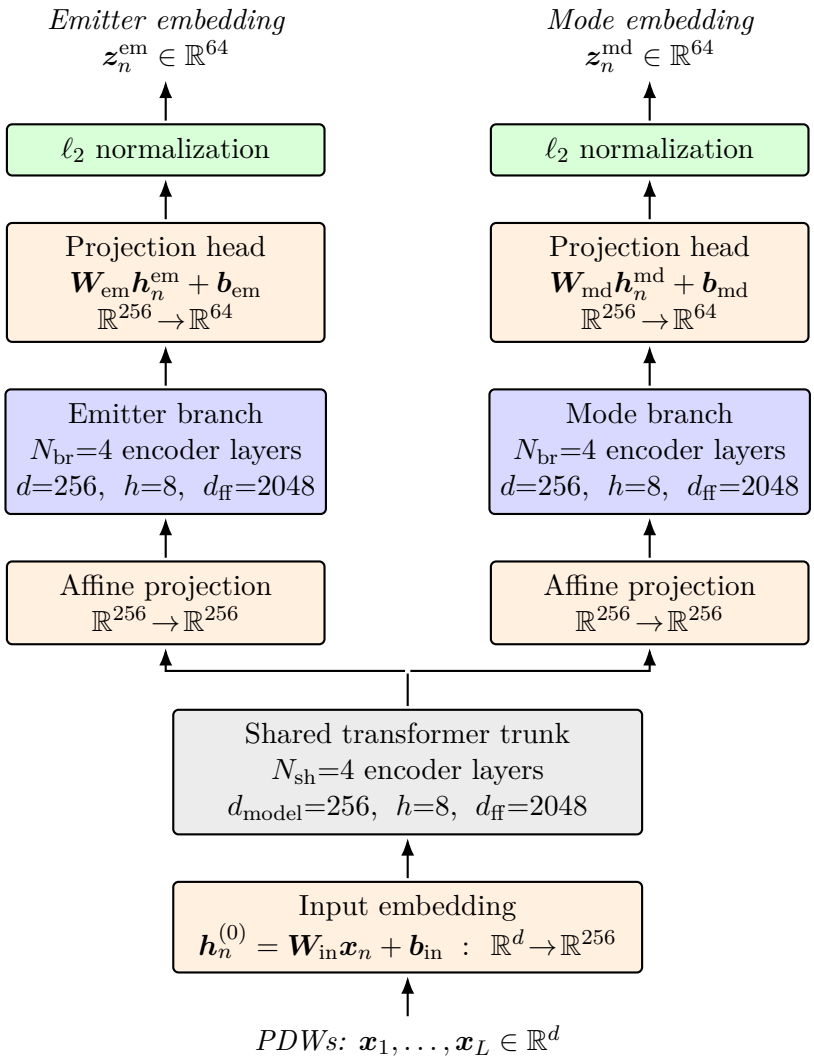


Figure 4.3. Architecture of the encoder $f_{\boldsymbol{\theta}}$. A sequence of PDWs is first lifted to $d_{\mathrm{model}} = 256$ by an affine input embedding, processed by a shared trunk of 4 transformer-encoder layers, and then split into two parallel branches, each with its own affine $256 \to 256$ projection and 4 encoder layers. Branch outputs are mapped by affine projection heads to the 64-dimensional emitter and mode embedding spaces and ℓ_2-normalized over the embedding dimension. The input embedding, branch projections, and projection heads are all affine maps of the same form.

Input embedding

Each pulse vector $\boldsymbol{x}_n \in \mathbb{R}^d$, built from the scaled PDWs of Section 4.2, is mapped to a token embedding through an affine projection

$$\boldsymbol{h}_n^{(0)} = \boldsymbol{W}_{\text{in}}\boldsymbol{x}_n + \boldsymbol{b}_{\text{in}}, \qquad \boldsymbol{h}_n^{(0)} \in \mathbb{R}^{d_{\text{model}}}, \tag{4.11}$$

where $\boldsymbol{W}_{\text{in}} \in \mathbb{R}^{d_{\text{model}} \times d}$ and d_{model} is the trunk width specified below. In the baseline configuration of the encoder, no positional encoding is added to $\boldsymbol{h}_n^{(0)}$. The temporal ordering of pulses is already carried by the TOA entry of $\boldsymbol{x}_n$; moreover, the index-based absolute positional encodings of the original architecture [12] are ill-suited to this input, since the TOA gap between consecutive pulses is not constant and an ordinal position therefore misrepresents the relative distances in time. Whether an explicit relative encoding of time and arrival direction nevertheless adds information beyond their presence as input features is revisited in Sections 4.4.1 and 4.4.2, where a pairwise attention bias and a rotary positional encoding on physical coordinate differences are used as drop-in modifications of this architecture, including both mechanisms together.

Shared trunk

The token sequence $(\boldsymbol{h}_1^{(0)}, \ldots, \boldsymbol{h}_L^{(0)})$ is processed by a stack of N_{sh} standard transformer-encoder layers, each comprising a MHA sub-layer and a position-wise FFN sub-layer with residual connections and layer normalisation around both. Self-attention is unmasked, so every pulse can attend to every other pulse in the window. The trunk operates with hidden dimension $d_{\text{model}} = 256$, $h_{\text{sh}} = 8$ attention heads (so that each head has dimension 32), and an inner FFN dimension of $d_{\text{ff,sh}} = 2048$; $N_{\text{sh}} = 4$ layers are used. Its output is a contextualised token sequence $\boldsymbol{H}^{\text{sh}} = (\boldsymbol{h}_1^{\text{sh}}, \ldots, \boldsymbol{h}_L^{\text{sh}})$ with $\boldsymbol{h}_n^{\text{sh}} \in \mathbb{R}^{256}$, in which information has been mixed across the full window.

Task-specific branches

The trunk output feeds two parallel branches that do not share parameters. Each branch begins with an affine projection of the same form as Equation (4.11), after which $N_{\text{br}} = 4$ transformer-encoder layers of the same form as the trunk are applied. The projection decouples the branch representation from the shared trunk output and, more generally, allows the branch width to differ from the trunk width; here the branches retain $d_{\text{br}} = 256$, so the projection maps $\mathbb{R}^{256} \to \mathbb{R}^{256}$. Within a branch, MHA uses $h_{\text{br}} = 8$ heads (per-head dimension 32) and the FFN inner dimension is $d_{\text{ff,br}} = 2048$. The incidence rotary variant of Section 4.4.2 is the one exception: it uses 4 heads on the deinterleaving branch so that the two incidence axes can split a 64-dimensional head evenly. Write the resulting token sequences as $\boldsymbol{H}^{\text{em}} = (\boldsymbol{h}_1^{\text{em}}, \ldots, \boldsymbol{h}_L^{\text{em}})$ and $\boldsymbol{H}^{\text{md}} = (\boldsymbol{h}_1^{\text{md}}, \ldots, \boldsymbol{h}_L^{\text{md}})$ with $\boldsymbol{h}_n^{\text{em}}, \boldsymbol{h}_n^{\text{md}} \in \mathbb{R}^{256}$.

Projection heads

Each branch terminates in an affine projection head—of the same form as the input embedding Equation (4.11)—that maps every token to its respective embedding space, followed by ℓ_2 normalization over the embedding dimension,

$$\boldsymbol{z}_n^{\mathrm{em}} = \frac{\boldsymbol{W}_{\mathrm{em}}\boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}}{\left\|\boldsymbol{W}_{\mathrm{em}}\boldsymbol{h}_n^{\mathrm{em}} + \boldsymbol{b}_{\mathrm{em}}\right\|_2}, \qquad \boldsymbol{z}_n^{\mathrm{md}} = \frac{\boldsymbol{W}_{\mathrm{md}}\boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}}{\left\|\boldsymbol{W}_{\mathrm{md}}\boldsymbol{h}_n^{\mathrm{md}} + \boldsymbol{b}_{\mathrm{md}}\right\|_2}, \tag{4.12}$$

with $\boldsymbol{z}_n^{\mathrm{em}}, \boldsymbol{z}_n^{\mathrm{md}} \in \mathbb{R}^p$ for a common embedding dimension $p = 64$. Collecting the per-pulse outputs gives the two embedding sequences

$$\boldsymbol{Z}^{\mathrm{em}} = (\boldsymbol{z}_1^{\mathrm{em}}, \ldots, \boldsymbol{z}_L^{\mathrm{em}}), \qquad \boldsymbol{Z}^{\mathrm{md}} = (\boldsymbol{z}_1^{\mathrm{md}}, \ldots, \boldsymbol{z}_L^{\mathrm{md}}). \tag{4.13}$$

The normalization constrains both embedding sequences to the unit sphere in $\mathbb{R}^{64}$, so cluster structure is expressed through angular separation rather than vector magnitude; this is the geometry assumed by the cosine-similarity contrastive objective of Section 4.5.

From continuous embeddings to discrete partitions

The projection heads in Equation (4.12) produce continuous vectors, whereas the per-pulse outputs in Equation (4.4) are discrete. At inference time, each branch's L-token embedding sequence is therefore decoded independently and per window by a fixed clustering operator,

$$(\hat{e}_1, \ldots, \hat{e}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{em}}\}_{n=1}^{L}\Big), \quad (\hat{m}_1, \ldots, \hat{m}_L) = \mathrm{DBSCAN}\Big(\{\boldsymbol{z}_n^{\mathrm{md}}\}_{n=1}^{L}\Big), \tag{4.14}$$

with cardinalities $\hat{K}, \hat{M} \in \mathbb{N}$ inferred from the data (Figure 4.4). DBSCAN [4] groups embeddings by local density at a Euclidean radius ε and minimum neighbourhood count minPts, assigning sparser points to a noise category. The radius ε is selected by a grid search over $\varepsilon \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6\}$ on the validation set, separately for each branch, using the clustering metrics of Section 4.7; the selected values are reported in Chapter 5. The neighbourhood count is fixed at minPts $= 5$, following its use in prior work on embedding-based deinterleaving [21]. With these settings the noise category remained marginal on the validation set—empty for most windows and almost never exceeding ten pulses per window—so no further treatment of noise-labelled pulses was deemed necessary. The clustering operator is applied only at inference, independently on each window and on each branch, and contributes no gradients. Cluster indices are therefore local to that window; the global mode labels of Section 4.1 enter only through the training loss. The trainable parameters $\boldsymbol{\theta}$ comprise the trunk, branch, and projection-head weights and are optimised by the loss of Section 4.5.

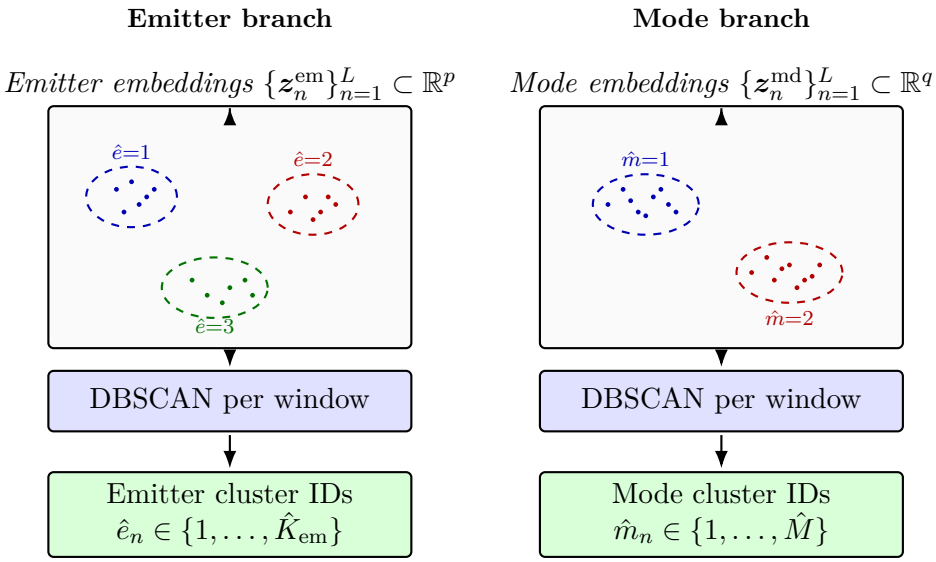


Figure 4.4. Inference-time decoding of branch embeddings into per-pulse cluster indices via Equation (4.14). Dots are pulse embeddings and dashed ellipses are DBSCAN clusters in one window. The pipeline is identical on both branches and the integer outputs are local to each clustering instance; they acquire their interpretation from the supervision scope of Section 4.1, which differs between the two branches.

4.4 Attention Variants

The encoder of Section 4.3 uses standard self-attention, which treats a window as an unordered set of tokens and lets every pulse attend to every other pulse purely through learned query–key compatibility. The only ordering cue available to it is whatever the input features themselves carry, in particular the TOA. A radar recording, however, has strong pairwise structure: two pulses that are close in time, share a carrier frequency, have a similar pulse width, or arrive from a similar direction, are far more likely to originate from the same emitter or mode than an arbitrary pair. This section describes two variants of the attention mechanism that make such structure explicit. They are used as drop-in modifications of the MHA sub-layers of Section 4.3. Each mechanism is evaluated on its own against standard self-attention, and they are also stacked: rotary encoding on window-local TOA in every layer of the trunk and of both branches, together with the pairwise bias, with no incidence rotary coordinates. All of these layouts are included in Chapter 5.

Throughout, we write the attention logits of a single head over a window of L pulses as

$$\boldsymbol{A} = \frac{\boldsymbol{Q}\boldsymbol{K}^{\top}}{\sqrt{d_k}}, \qquad A_{ij} = \frac{\boldsymbol{q}_i^{\top}\boldsymbol{k}_j}{\sqrt{d_k}}, \tag{4.15}$$

where $\boldsymbol{q}_i$ and $\boldsymbol{k}_j$ are the query and key of pulses i and j and d_k is the per-head dimension. The attention weights are the row-wise softmax of $\boldsymbol{A}$, which then mixes the value vectors.

4.4.1 Physical attention bias

The first variant adds a pairwise bias to the attention logits, computed from the physical parameters of each pulse pair. The implemented coordinates are the window-local TOA t_n and the two planar Euclidean incidence components $(\tilde{u}_n^x, \tilde{u}_n^y)$ of Equation (4.8). None of these coordinates is a raw angle. Each selected coordinate p contributes an absolute pairwise difference $B_{ij}^{(p)} = |p_i - p_j|$, added to the logits with its own learnable scalar coefficient. On the shared trunk only TOA is used,

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + \lambda_t \, B_{ij}^{(t)}. \tag{4.16}$$

On the two task branches all three coordinates are present,

$$A_{ij} = \frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d_k}} + \lambda_t \, B_{ij}^{(t)} + \lambda_x \, B_{ij}^{(x)} + \lambda_y \, B_{ij}^{(y)}, \tag{4.17}$$

where $B^{(x)}$ and $B^{(y)}$ are the absolute differences of $\tilde{u}^x$ and $\tilde{u}^y$. Each coefficient is learned jointly with the rest of the network and belongs to a single head of a single attention sub-layer. This adds one scalar per head, layer, and selected coordinate; the pairwise delta matrices themselves scale quadratically in the window length, as the attention mechanism already does. Coefficients may be negative, and each is initialised to a small negative value so that, in line with the intended inductive bias, a larger coordinate difference initially decreases attention between the corresponding pulses. Because the coefficients are learned independently in each part of the network, the branches can weight the cues differently. The deinterleaving branch is expected to rely more on incidence, since pulses from one emitter usually arrive from a nearly constant direction. The shared trunk and the mode branch should instead suppress that cue, because an operating mode must be recognised independently of viewing geometry. Whether this allocation emerges is examined in Section 5.6.4. The bias follows the same idea as additive pairwise attention biases in relative-position and graph Transformers [31, 32], but uses continuous pulse parameters instead of discrete token or path distances.

4.4.2 Rotary positional encoding

The bias of Equation (4.17) is added to the logits after the query–key product. The second variant uses the rotary encoding of Section 2.7, so the inner product depends on a pulse coordinate only through pairwise differences (Equation (2.23)).

In the original construction the coordinate is the token index. Consecutive pulses in a recording are not equally spaced, so an ordinal index misrepresents TOA gaps (Section 4.2). We attach physical coordinates instead, so that $p_j - p_i$ is a time gap or a difference of direction features.

Those coordinates lie in $[0, 1]$ after the maps of Section 4.2. A token index over the same window would instead run through $\{0, \ldots, L-1\}$ in steps of one. Feeding the unit

interval to $\boldsymbol{R}(p)$ with the original frequencies would leave almost every plane unrotated: the first plane would turn by at most one radian from the start of the window to the end, and the slow planes by about $1/b$. A scale $\gamma > 0$ therefore stretches p so that plane m is rotated by $\gamma p \omega_m$. Equivalently, $\boldsymbol{R}$ is evaluated at γp. The factor is left implicit in $\boldsymbol{R}$ below.

The base is kept at $b = 10^4$ as in Su et al. [14]. Changing b would alter how quickly successive planes slow down, and therefore how large a gap the last plane can represent before wrapping. Changing γ multiplies every frequency, including the fastest, and maps the unit interval onto an effective index range of length γ. Choosing $\gamma = L$ would give that interval the same numeric span as token indices on a window of this length. A larger γ gives a small gap in p more phase and makes the fast planes wrap inside the window. A smaller γ keeps phases unique over a wider span of the window and flattens nearby distinctions. Unlike the bias coefficients of Equation (4.17), γ is not learned. A separate scale per coordinate would let time and direction be stretched independently, because a numerical gap of a given size is not the same physical distance in time as in direction. In every rotary layout they are nevertheless set equal, $\gamma_t = \gamma_x = \gamma_y = 10$. That is far below $\gamma = L$, so a unit gap in a normalised coordinate produces only a modest phase and the fastest planes do not wrap inside a window.

The encoder of Section 4.3 uses this construction in every attention sub-layer. The shared trunk and the mode branch rotate by the window-local TOA. The deinterleaving branch splits each head across the two planar Euclidean incidence components of Equation (4.8). It does not rotate by a scalar azimuth or elevation angle.

Trunk and mode branch

Every head in the shared trunk and in the mode branch rotates by the window-local TOA. With scale γ_t in $\boldsymbol{R}$, the attention logit of pulses i and j is

$$\left\langle \tilde{\boldsymbol{q}}_i,\, \tilde{\boldsymbol{k}}_j \right\rangle = \boldsymbol{q}_i^{\top}\, \boldsymbol{R}(t_j - t_i)\, \boldsymbol{k}_j. \tag{4.18}$$

Arrival direction is excluded. An operating mode is characterized by the intrinsic temporal pattern of the emission and must be recognized regardless of viewing geometry. Rotating these layers by direction would make the mode embedding depend on that geometry.

Deinterleaving branch

Every head in the deinterleaving branch rotates by planar incidence instead. Direction is a strong cue for emitter identity, and temporal structure is already present upstream through the TOA-rotated trunk. Feeding the raw azimuth θ_n^{az} into RoPE would reintroduce the 0–2π wrap of Section 4.2.1, because the relative phase of Equation (2.23) depends on the algebraic difference of the scalar coordinate. The branch therefore uses

the two Euclidean components already in the input,

$$\tilde{u}_n^x,\ \tilde{u}_n^y \in [0,1], \tag{4.19}$$

from Equation (4.8). These are Cartesian incidence coordinates, not angles. The $d_k/2$ planes of each head are split evenly into two slices of $d_k/4$ planes, following multi-axis RoPE [33], so that the query (and likewise the key) decomposes as

$$\boldsymbol{q}_n = \begin{bmatrix} \boldsymbol{q}_n^{(x)} \\ \boldsymbol{q}_n^{(y)} \end{bmatrix}, \qquad \boldsymbol{q}_n^{(x)},\, \boldsymbol{q}_n^{(y)} \in \mathbb{R}^{d_k/2}. \tag{4.20}$$

Each half is rotated by its own single-coordinate matrix of the form Equation (2.21),

$$\boldsymbol{R}_n^{\mathrm{em}} = \mathrm{diag}\Big(\boldsymbol{R}^{(x)}(\tilde{u}_n^x),\ \boldsymbol{R}^{(y)}(\tilde{u}_n^y)\Big), \tag{4.21}$$

with $\boldsymbol{R}^{(x)}, \boldsymbol{R}^{(y)} \in \mathbb{R}^{d_k/2 \times d_k/2}$ using scales γ_x and γ_y respectively. The two halves of the query are then

$$\tilde{\boldsymbol{q}}_n^{(x)} = \boldsymbol{R}^{(x)}(\tilde{u}_n^x)\, \boldsymbol{q}_n^{(x)}, \qquad \tilde{\boldsymbol{q}}_n^{(y)} = \boldsymbol{R}^{(y)}(\tilde{u}_n^y)\, \boldsymbol{q}_n^{(y)}, \tag{4.22}$$

and likewise for the key. The x rotation acts only on $\boldsymbol{q}_n^{(x)}$, the y rotation only on $\boldsymbol{q}_n^{(y)}$. Because the slices occupy disjoint coordinates, the head inner product is the sum of two half-dimension inner products,

$$\begin{aligned} \left\langle \tilde{\boldsymbol{q}}_i,\, \tilde{\boldsymbol{k}}_j \right\rangle &= \left\langle \tilde{\boldsymbol{q}}_i^{(x)},\, \tilde{\boldsymbol{k}}_j^{(x)} \right\rangle + \left\langle \tilde{\boldsymbol{q}}_i^{(y)},\, \tilde{\boldsymbol{k}}_j^{(y)} \right\rangle \\ &= \boldsymbol{q}_i^{(x)\top}\, \boldsymbol{R}^{(x)}\Big(\tilde{u}_j^x - \tilde{u}_i^x\Big)\, \boldsymbol{k}_j^{(x)} + \boldsymbol{q}_i^{(y)\top}\, \boldsymbol{R}^{(y)}\Big(\tilde{u}_j^y - \tilde{u}_i^y\Big)\, \boldsymbol{k}_j^{(y)}. \end{aligned} \tag{4.23}$$

Two arrivals that differ by a full turn in azimuth but point in the same direction have nearly the same $(\tilde{u}^x, \tilde{u}^y)$, so the relative phase in Equation (4.23) stays small. An algebraic gap in θ^{az} itself would not.

This branch uses 4 heads rather than 8, so each incidence axis occupies $d_k/4 = 16$ planes of a 64-dimensional head.

A control that rotates every layer of the trunk and of both branches by window-local TOA, with no incidence encoding, is evaluated in Chapter 5. That uniform layout tests whether the branch-specific allocation above is necessary, or whether a single temporal coordinate already suffices. The same uniform TOA layout is also used together with the pairwise bias of Equations (4.16) and (4.17).

Figure 4.5 summarizes the plane allocation for the trunk/mode and deinterleaving layouts.

4.5 Loss Function

The training objective is a sum of two supervised contrastive [11] terms with the same functional form, both aggregated over the whole mini-batch. The two terms differ

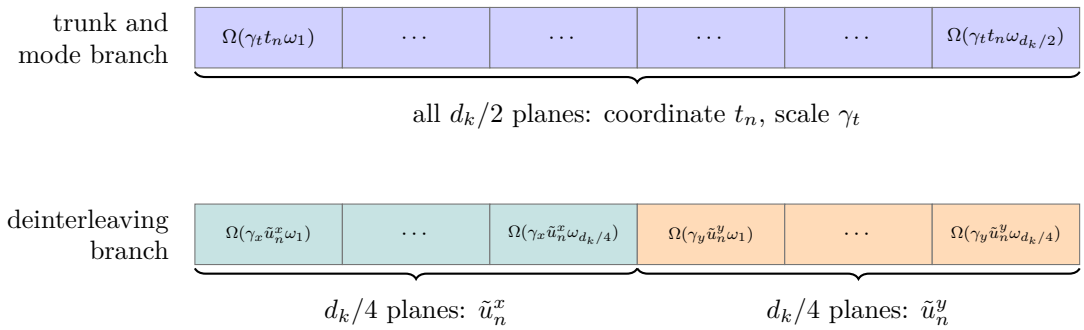


Figure 4.5. Allocation of the $d_k/2$ rotation planes of one attention head. Top: shared trunk and mode branch, all planes rotated by the TOA t_n with scale γ_t (Equation (4.18)). Bottom: deinterleaving branch, planes split evenly between the Euclidean incidence components $\tilde{u}_n^x$ and $\tilde{u}_n^y$ of Equation (4.8), with scales γ_x and γ_y (Equations (4.21) and (4.23)). Ellipsis cells are intermediate planes of each frequency ladder.

only in which label drives the positive set: an emitter identifier on the emitter branch, and the global mode label of Section 4.1 on the mode branch. Both branches use a contrastive loss rather than a closed-catalogue softmax, so training pushes pulses with the same label together in embedding space instead of scoring a fixed set of class logits. That encourages the encoder to learn general similarity structure among pulses. Inference then clusters those embeddings (Equation (4.14)); the clustering step is not part of the loss and contributes no gradients. In principle the same embeddings could also support open-set modes or a later classification head on a library of known emitters using the outputted encodings, but those uses are not evaluated in this thesis.

Supervised contrastive kernel

The projection-head outputs of Equation (4.12) are unit-norm by construction, so that the cosine similarity reduces to the inner product

$$s_{ij} \;\coloneqq\; \boldsymbol{z}_i^{\top} \boldsymbol{z}_j \;\in\; [-1, 1], \tag{4.24}$$

and a fixed temperature $\tau > 0$ rescales it inside the loss. Smaller τ concentrates the gradient on the hardest negatives, while larger τ stabilizes early optimization, since $\|\nabla \mathcal{L}_{\mathrm{SC}}\| \propto \frac{1}{\tau}$ [11]. Both branches use $\tau = 0.5$, the value suggested in the original supervised contrastive formulation [11].

Let a mini-batch consist of B windows of length L, and write $\mathcal{B}$ for the set of all $B \times L$ pulse embeddings pooled across those windows. How the windows are chosen is controlled by the scenario-aware sampler of Section 4.6. For each pulse $i \in \mathcal{B}$, write the candidate set $A(i) = \mathcal{B} \setminus \{i\}$ and the positive subset $P(i) \subseteq A(i)$ defined by label coincidence. The supervised contrastive contribution [11] is then

$$\mathcal{L}_{\mathrm{SC}} \;=\; -\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(s_{ip}/\tau)}{\sum_{a \in A(i)} \exp(s_{ia}/\tau)}, \tag{4.25}$$

where $\mathcal{B}^+ = \{i \in \mathcal{B} : P(i) \neq \emptyset\}$ and $\mathcal{L}_{\mathrm{SC}} = 0$ when $\mathcal{B}^+ = \emptyset$. Pulses outside $\mathcal{B}^+$ contribute only as negatives. The denominator runs over every other pulse in the batch, so unrelated samples exert repulsive pressure and the encoder cannot collapse to a constant map [10].

Branch losses and total objective

Both branches apply Equation (4.25) to the same batch embeddings and differ only in the label that defines the positive sets: two pulses are positives on the emitter branch when they originate from the same physical emitter, and on the mode branch when they carry the same global mode label. Since emitter identifiers are recording-local in Section 4.1, they are made unique within a mini-batch by pairing each identifier with its originating recording. Writing $\mathcal{L}_{\mathrm{em}}$ and $\mathcal{L}_{\mathrm{md}}$ for the resulting losses on the emitter and mode projections, the total training objective is

$$\mathcal{L} \;=\; \mathcal{L}_{\mathrm{em}} + \lambda_{\mathrm{md}}\,\mathcal{L}_{\mathrm{md}}, \qquad \lambda_{\mathrm{md}} = 1, \tag{4.26}$$

so the two branches carry equal weight. Tuning λ_{md} could improve the trade-off between the tasks, but is not evaluated in this thesis. The factor $1/|P(i)|$ in Equation (4.25) keeps each pulse's contribution intensive in its positive-set size, which matters because emitter positives are usually few while mode positives can be many; a Gibbs–Boltzmann reading of the same kernel is given in Chapter A.

4.6 Training Procedure

Parameters $\boldsymbol{\theta}$ are estimated by minimising $\mathcal{L}$ (Equation (4.26)) with AdamW [36] in PyTorch. Training runs for ten epochs on the training split; after each epoch the model is evaluated on the validation split. The learning rate rises linearly from zero to its peak over the first 5% of the total optimisation steps (ten epochs times the number of training batches), then follows a cosine anneal. Peak learning rate, weight decay, batch size, and related settings are listed in Table 5.2.

Scenario-aware batch sampling

Mini-batches are not formed by uniform shuffling of the pooled windows. Emitter identifiers are recording-local (Section 4.1), so two windows drawn from different scenarios never share an emitter positive even when both contain pulses from physically similar sources. Uniform mixing across the corpus would therefore leave the emitter branch with positives only inside each individual window, unless two windows happen to be drawn from the same scenario—an event that is unlikely among roughly 1500 training scenarios (Section 5.2). Such sparse positives can hinder learning and leave most of a multi-window batch unused for the emitter term.

Each mini-batch is instead drawn from a single scenario, with several windows taken from that scenario so that the same emitter can appear across windows and supply additional positives. Mode labels remain globally comparable across scenarios, but drawing several windows from one scenario also increases the number of same-mode positives available to the mode branch in each batch. The number of windows per batch is recorded in Table 5.2.

4.7 Evaluation Metrics

Inference returns, for each window, an emitter labelling $(\hat{e}_1, \ldots, \hat{e}_L)$ and a mode labelling $(\hat{m}_1, \ldots, \hat{m}_L)$ (Equation (4.14)). These are compared with the simulator labels (e_n, m_n) of Equation (4.1) on the same window. Cluster names are assigned independently per window and per branch, so only scores that are invariant to relabelling are valid. The definitions are those of Section 2.5.

Each branch is scored separately: emitter agreement from $(e_n, \hat{e}_n)$ and mode agreement from $(m_n, \hat{m}_n)$. Global mode labels enter training as an inductive bias that pulls same-mode embeddings together. They are not a catalogue of names at inference, and the mode score remains a partition comparison.

The reported scores are ARI, AMI, homogeneity, and completeness, computed on every test window. ARI summarises pair agreement and stays comparable when the number of active emitters or modes changes between windows. AMI measures how much one labelling tells about the other. It is less dominated by the largest clusters. Pulse counts per emitter are often unbalanced, so that difference is useful here. Homogeneity and completeness are reported separately rather than only through the V-measure. A high homogeneity with low completeness means true emitters or modes were split; the reverse means distinct sources were merged.

Two-dimensional projections of $\{\boldsymbol{z}_n^{\mathrm{em}}\}$ and $\{\boldsymbol{z}_n^{\mathrm{md}}\}$, coloured by (e_n, m_n), show whether same-label pulses sit together in each embedding. Protocols and numbers are in Chapter 5.

4.8 Baseline Methods

The comparisons isolate the contribution of the mode term $\lambda_{\mathrm{md}}\mathcal{L}_{\mathrm{md}}$ in Equation (4.26) from that of the attention variants in Section 4.4, relative to standard self-attention. Final configurations and numbers are deferred to Chapter 5; this section fixes only the design intent of each control.

Dual-branch, single-task. The encoder of Section 4.3 with standard self-attention, trained with one contrastive term only: $\mathcal{L}_{\mathrm{em}}$ (emitter-only) or $\mathcal{L}_{\mathrm{md}}$ (mode-only). Architecture, width, and decoding remain those of the joint model, so the only change is the objective. Emitter-only is the comparable transformer for whether the

joint loss outperforms deinterleaving-alone training. Mode-only is the symmetric control, trained only on $\mathcal{L}_{\text{md}}$. Both branches are still decoded with DBSCAN, including the branch that received no direct gradient.

DBSCAN on fixed window features. No transformer; DBSCAN [4] clusters a deterministic vector built from the scaled PDWs of a window. There is no training. The neighbourhood radius ε is selected by grid search on the validation set, holding minPts fixed; the selected ε is then frozen for the held-out test evaluation. Separates the gain from learned sequence context from what density clustering already recovers on hand-specified summaries.

CHAPTER 5

Experiments and Results

5.1 Experimental Setup

All neural models in this chapter were trained and evaluated on a single workstation with one NVIDIA A100 GPU (80 GB high-bandwidth memory). The implementation is written in Python with PyTorch. The source code and training scripts remain proprietary to Saab and are not released with this thesis.

5.2 Datasets

All experiments in this chapter use synthetic PDW streams generated by the proprietary scenario simulator introduced in Section 4.2. The simulator supplies time-ordered pulse records together with ground-truth labels for emitter identity (meaningful only within one scenario, in the sense of Section 4.1) and operating mode (drawn from the global catalogue); both label streams are consumed by the training objective in Section 4.5 and are also used to compute the evaluation metrics in Sections 4.7 and 5.4.

Each *scenario* is one simulated timeline: a sequence of PDWs whose length and the number of concurrent emitters vary across draws so that the encoder sees both sparse and crowded electromagnetic pictures. Scenarios are disjointly assigned to training, validation, and test partitions before any windowing; training-split statistics for carrier frequency, pulse width, and amplitude in Section 4.2 are fit on the training partition only and applied unchanged to validation and test pulses, whereas TOA is min–max mapped independently inside each window. Windows contain $L = 2000$ pulses. The neural run configuration uses stride $\delta = 200$ on every split, so consecutive dumped windows overlap by $L - \delta = 1800$ pulses. Training consumes that overlapping stream. Reported validation and test scores keep every tenth dumped window, which restores $\delta = L$ and yields non-overlapping windows (Section 5.4.1). Feature DBSCAN of Section 4.8 is not trained; its test dump uses the same L with overlap already 0.

Table 5.1 summarises corpus-level figures. The training split contains 1461 scenarios and roughly 13.8 million pulses in total, while the validation and test splits contribute approximately 2.25 and 2.36 million pulses over 245 and 244 scenarios, respectively. The emitter column counts the distinct emitters active in each scenario. Emitter identifiers are meaningful only within their own scenario (Section 4.1), so these figures describe how many emitters each scenario contains rather than membership in some global emitter set; at inference, the within-window clustering accordingly never has to separate more emitters than the enclosing scenario contains. The number of unique emitters

Table 5.1. Synthetic scenario corpus used for training and evaluation. Per-scenario quantities are reported as mean ± standard deviation over the scenarios in each split.

		Per scenario (mean ± std)		
Split	Scenarios	Pulses	Emitters	Modes
Train	1461	9480 ± 1690	8.80 ± 4.33	4.33 ± 1.40
Validation	245	9180 ± 2240	8.59 ± 4.41	4.28 ± 1.57
Test	244	9670 ± 1270	8.93 ± 4.56	4.39 ± 1.49

per scenario ranges from 1 to 23 in every split, so the three partitions are statistically homogeneous in this respect. A window of length L never contains that many concurrent emitters: on the non-overlapping test windows used for scoring, the largest number of distinct emitter labels in one window is 7. The global mode catalogue is taken to have size $M = 16$, equal to the largest number of distinct mode labels in any of those same evaluated windows. A recording uses a subset of that catalogue. The catalogue is shared across the training, validation, and test splits.

The numbers of emitters and assigned modes per scenario are sampled uniformly; however, the resulting pulse counts per emitter and mode are highly imbalanced due to variation in mode behavior. For instance, scanning modes typically generate far fewer pulses than lock-on modes, and some modes differ by more than an order of magnitude in their base PRI. As a result, the number of pulses (PDWs) per emitter or mode is not balanced either within a scenario or across the corpus. Because the encoder sees only windows of fixed length L (Section 4.2), rare combinations of emitter, mode, and interleaving pattern can be missing from individual windows or represented by only a handful of pulses, which limits what any single forward pass can evidence about those labels.

5.2.1 Qualitative properties of the synthetic corpus

Beyond aggregate counts, the corpus is checked qualitatively before large training runs: example timelines are inspected for plausible PRI structure per mode, for physically admissible RF and pulse-width ranges, and for the intended mixture density when multiple emitters are active. When the simulator regime adds dropped or spurious pulses (Section 4.2), spot checks confirm that surviving pulses retain their original time order and that spurious insertions stay within the configured parameter ranges. This mirrors the reporting practice in earlier Saab-hosted simulator studies, which often include illustrative emitter tracks alongside tables [19]. Section 5.7 inspects one light window and one crowded window from the test dump after training.

Table 5.2. Primary optimisation hyperparameters for the proposed model.

Setting	Value
Optimiser	AdamW [36] with PyTorch defaults $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$
Peak learning rate $\eta_{\max}$	5×10^{-4}
Warmup	Linear from 0 to $\eta_{\max}$ over 5% of total steps
Schedule after warmup	Cosine annealing to $\eta_{\min}$
Minimum learning rate $\eta_{\min}$	0
Epochs	10
Window length L	2000
Training stride δ	200 (overlap 1800 pulses)
Dropout (attention and feed-forward)	0.1 throughout the transformer blocks
Batch size B (windows per step)	4
Mode loss weight λ_{md}	1
Weight decay	0.01 (PyTorch AdamW default)

5.3 Hyperparameters and Training Details

This section collects numerical training choices deferred from Sections 4.3, 4.5 and 4.6. Unless Section 5.5 states otherwise, jointly trained encoders use the same epoch budget and hardware (Section 5.1) so that differences reflect architecture and objective rather than compute. Feature DBSCAN is not trained.

The contrastive temperature is fixed at $\tau = 0.5$ as in Section 4.5. Table 5.2 summarises the optimisation schedule and regularisation for the proposed dual-branch encoder.

Total optimisation steps equal ten epochs times the number of training batches per epoch. The warmup fraction and cosine phase are counted in those steps, as stated in Section 4.6.

The rotary scales are those of Section 4.4 and are not repeated here. Table 5.3 lists only the DBSCAN radii of Section 4.3.

The single-task runs of Section 5.5 inherit the Vanilla architecture and the optimisation schedule of Table 5.2. Their radii are selected independently, as for the joint models.

5.4 Quantitative Results

This section states the evaluation protocol used in the rest of the chapter and reports the existence test for RQ1: a jointly trained Vanilla encoder against Feature DBSCAN. The joint-versus-single-task comparison (RQ2) follows in Section 5.5. The remaining jointly trained encoders (RQ3) are compared in Section 5.6.

Table 5.3. DBSCAN radii. ε_{em} and ε_{md} are selected independently per model on the validation set; minPts = 5 is fixed.

Model	ε_{em}	ε_{md}
Feature DBSCAN	0.1	0.1
Vanilla	0.6	0.1
RoPE-TOA	0.6	0.1
RoPE-az	0.6	0.2
Bias	0.6	0.1
RoPE-TOA+Bias	0.3	0.1
Emitter-only	0.6	0.1
Mode-only	0.5	0.1

5.4.1 Evaluation protocol

All scores are computed on the held-out test split of Section 5.2. Every model is scored with window length $L = 2000$ and with no overlap between scored windows. The neural evaluation dumps used stride 200; every tenth window is kept here, which restores stride $\delta = L$ and yields 960 non-overlapping windows over the 244 test scenarios. Feature DBSCAN of Section 4.8 clusters the scaled PDW vector of each pulse directly. It is not trained, and its test dump already uses overlap 0 at the same L, so no subsample is applied. For each window, DBSCAN on each branch yields a predicted partition that is scored against the simulator labels with ARI, AMI, homogeneity, and completeness (Section 4.7).

Each neural table entry is the mean ± one standard deviation of that per-window score over the 960 windows. The DBSCAN radius ε is selected independently for each model and each branch on the validation grid of Section 4.3, with minPts = 5 throughout; selected radii are listed in Table 5.3.

Homogeneity and completeness are reported separately rather than only through their harmonic mean, the V-measure, so that oversplitting can be distinguished from undermerging. Mean absolute cluster-count error $|\hat{K} - K|$ (emitters) and $|\hat{M} - M|$ (modes) summarises how well the inferred number of clusters matches the number of distinct reference labels in the same window.

5.4.2 Joint Vanilla encoder

Vanilla is the encoder of Section 4.3 with standard MHA, trained with the joint objective of Equation (4.26). TOA enters only as an input feature. Feature DBSCAN is the no-encoder control of Section 4.8. Table 5.4 reports both branches on the same windows, so the two tasks can be compared before any attention variant is introduced.

Feature DBSCAN sits well below Vanilla on both branches. Emitter ARI is 0.578, and 71% of windows fall below 0.90. Completeness is 0.523 and the mean count error exceeds seven emitters, so the method oversplits. Homogeneity remains 0.976: the extra

Table 5.4. Feature DBSCAN versus jointly trained Vanilla on the test split. Entries are mean ± std over 960 non-overlapping windows of length $L = 2000$. Feature DBSCAN is not trained. Hom. and Comp. are homogeneity and completeness. Count error is $|\hat{K} - K|$ on the emitter branch and $|\hat{M} - M|$ on the mode branch.

Model	Branch	ARI	AMI	Hom.	Comp.	Count error
Feature DBSCAN	Emitter	0.578 ± 0.348	0.630 ± 0.260	0.976 ± 0.049	0.523 ± 0.275	7.04 ± 5.85
Feature DBSCAN	Mode	0.811 ± 0.250	0.829 ± 0.196	0.962 ± 0.064	0.774 ± 0.218	4.99 ± 3.99
Vanilla	Emitter	0.944 ± 0.182	0.955 ± 0.142	0.989 ± 0.054	0.948 ± 0.172	0.25 ± 0.48
Vanilla	Mode	0.986 ± 0.100	0.987 ± 0.094	0.993 ± 0.037	0.990 ± 0.092	0.23 ± 0.50

clusters are internally pure. Mode ARI is 0.811, still below Vanilla, with a mean count error of five modes.

Vanilla already recovers both partitions on this corpus, which is the existence test of RQ1. Emitter ARI is 0.944, and 87% of windows score at or above 0.99, but about one window in ten falls below 0.90. That tail is what widens the standard deviation. Mode recovery is stronger: mean ARI 0.986, with only 2.3% of windows below 0.90. The two tasks are therefore not equally hard for this encoder. Deinterleaving is the branch that still fails on a minority of windows; mode clustering is already tight on almost every window. The per-window emitter tail is the Vanilla curve of Figure 5.1; the corresponding count errors are the Vanilla panel of Figure 5.3.

5.5 Ablation Studies

This section isolates the joint objective of Equation (4.26) from the attention mechanism. The runs use the Vanilla encoder of Section 4.3, so the only change is which contrastive term is active. That matched dual-branch control is the comparison for RQ2. The jointly trained attention variants are compared afterwards in Section 5.6.

The three objectives are:

Joint $\mathcal{L} = \mathcal{L}_{\text{em}} + \lambda_{\text{md}}\mathcal{L}_{\text{md}}$ with $\lambda_{\text{md}} = 1$, identical to Vanilla in Section 5.4.2.

Emitter-only $\mathcal{L} = \mathcal{L}_{\text{em}}$. The mode branch receives no direct gradient.

Mode-only $\mathcal{L} = \mathcal{L}_{\text{md}}$. The emitter branch receives no direct gradient.

Joint numbers in Table 5.5 are copied from the Vanilla rows of Table 5.4, not from a second training run. The single-task models use the same splits and the remaining hyperparameters of Section 5.3.

Both branches are decoded with DBSCAN in every run, including the branch that was not trained. The informative cells are therefore the emitter scores of Mode-only and the mode scores of Emitter-only. They measure how much the shared trunk transfers to the unsupervised branch, versus collapse of that branch.

Count error is $|\hat{K} - K|$ on the emitter branch and $|\hat{M} - M|$ on the mode branch.

Table 5.5. Joint versus single-task objectives on the Vanilla encoder. Aggregation as in Table 5.4. Hom. and Comp. are homogeneity and completeness. Joint entries are those of Vanilla, not a repeated run.

Objective	Branch	ARI	AMI	Hom.	Comp.	Count error
Joint	Emitter	0.944 ± 0.182	0.955 ± 0.142	0.989 ± 0.054	0.948 ± 0.172	0.25 ± 0.48
Joint	Mode	0.986 ± 0.100	0.987 ± 0.094	0.993 ± 0.037	0.990 ± 0.092	0.23 ± 0.50
Emitter-only	Emitter	0.999 ± 0.010	0.997 ± 0.012	0.996 ± 0.019	0.999 ± 0.008	0.09 ± 0.32
Emitter-only	Mode	0.815 ± 0.255	0.864 ± 0.167	0.793 ± 0.212	0.999 ± 0.013	2.13 ± 1.95
Mode-only	Emitter	0.712 ± 0.311	0.757 ± 0.237	0.931 ± 0.156	0.699 ± 0.261	1.85 ± 1.53
Mode-only	Mode	0.984 ± 0.105	0.985 ± 0.099	0.991 ± 0.038	0.989 ± 0.097	0.23 ± 0.51

On the emitter branch, Emitter-only outperforms Joint. Mean ARI rises from 0.944 to 0.999, the spread collapses, and the count error falls to 0.09 ± 0.32. Adding the mode term therefore costs deinterleaving accuracy on this backbone, at least at $\lambda_{\text{md}} = 1$.

The unpaid cost of dropping the mode term is the mode branch itself. Under Emitter-only that branch still produces clusters, but they merge: homogeneity falls to 0.79 while completeness stays at 0.999, and the mean count error exceeds two modes per window. The clusters are internally pure and too few.

Mode-only is the other single-task ceiling. Its mode ARI is 0.984, within rounding of Joint. The untrained emitter branch splits rather than merges: homogeneity stays at 0.93 while completeness falls to 0.70, and the mean count error is 1.85 ± 1.53. True emitters are broken into extra clusters. Window 100 is a local case, with two emitters recovered as four.

RQ2, as stated, asks whether the joint objective outperforms a comparable transformer trained for deinterleaving alone. On the emitter metrics it does not. It also does not beat Mode-only on the mode metrics. What it buys is that both partitions remain usable in the same run. No extra distribution figures are included here. The Joint per-window distributions are those of Vanilla in Figures 5.1 to 5.4. The next section asks whether physical structure in attention recovers the emitter accuracy of Emitter-only without dropping the mode term.

5.6 Attention Variants

Five configurations are trained with the joint objective of Equation (4.26). Hyperparameters follow Section 5.3. Vanilla is the joint encoder of Section 5.4.2. RoPE-TOA applies rotary encoding on the window-local TOA in the trunk and in both branches. RoPE-az is the branch-specific rotary design of Section 4.4. Despite the name, it does not rotate by azimuth or elevation: the trunk and the mode branch rotate by window-local TOA, and the deinterleaving branch rotates by the two planar Euclidean incidence components $\tilde{u}^x, \tilde{u}^y$ of Equation (4.8). Bias adds the pairwise physical logits of Equations (4.16) and (4.17). RoPE-TOA+Bias uses that same TOA rotary encoding in

Table 5.6. Emitter clustering on the test split for jointly trained attention variants. Entries are mean ± std over the same 960 windows as Table 5.4.

Model	ARI	AMI	Homogeneity	Completeness	$\lvert\hat{K} - K\rvert$
Vanilla	0.944 ± 0.182	0.955 ± 0.142	0.989 ± 0.054	0.948 ± 0.172	0.25 ± 0.48
RoPE-TOA	$\mathbf{1.000 \pm 0.003}$	0.998 ± 0.007	$\mathbf{0.997 \pm 0.013}$	$\mathbf{1.000 \pm 0.000}$	0.09 ± 0.31
RoPE-az	0.972 ± 0.136	0.976 ± 0.113	0.994 ± 0.045	0.973 ± 0.131	0.14 ± 0.37
Bias	$\mathbf{1.000 \pm 0.003}$	$\mathbf{0.999 \pm 0.006}$	$\mathbf{0.997 \pm 0.012}$	$\mathbf{1.000 \pm 0.000}$	$\mathbf{0.09 \pm 0.30}$
RoPE-TOA+Bias	0.853 ± 0.253	0.888 ± 0.192	0.982 ± 0.078	0.851 ± 0.231	0.69 ± 0.90

Table 5.7. Mode clustering on the test split. Entries follow the same aggregation as Table 5.6.

Model	ARI	AMI	Homogeneity	Completeness	$\lvert\hat{M} - M\rvert$
Vanilla	0.986 ± 0.100	0.987 ± 0.094	0.993 ± 0.037	0.990 ± 0.092	0.23 ± 0.50
RoPE-TOA	0.990 ± 0.087	0.989 ± 0.083	0.995 ± 0.023	0.990 ± 0.084	0.20 ± 0.49
RoPE-az	0.984 ± 0.097	0.987 ± 0.084	0.989 ± 0.047	0.993 ± 0.079	0.23 ± 0.50
Bias	$\mathbf{0.993 \pm 0.074}$	$\mathbf{0.993 \pm 0.073}$	$\mathbf{0.997 \pm 0.017}$	$\mathbf{0.994 \pm 0.073}$	$\mathbf{0.13 \pm 0.39}$
RoPE-TOA+Bias	0.982 ± 0.104	0.983 ± 0.096	0.990 ± 0.037	0.986 ± 0.097	0.40 ± 0.75

the trunk and in both branches, together with the pairwise bias. Incidence is not used as a rotary coordinate.

The rows other than Vanilla test whether explicit physical structure in attention improves clustering relative to standard MHA (RQ3). Feature DBSCAN remains the no-encoder control of Table 5.4; it is omitted here. Tables 5.6 and 5.7 collect the window-averaged scores. The best value in each column is in bold; ties are bold on every tied row.

RoPE-TOA and Bias remove the emitter tail that joint Vanilla leaves in Tables 5.4 and 5.5. Mean emitter ARI is 1.000 to three decimals, 99% of windows sit at or above 0.99, and none of the 960 windows falls below 0.90. Completeness is 1.000 with no spread, so a true emitter is not split across clusters. The leftover errors are rare undercounts, visible as a small homogeneity gap and as the count-error column. Emitter-only Vanilla already reaches this emitter range in Table 5.5, without a usable mode branch. RoPE-TOA and Bias reach it under the joint objective.

RoPE-az improves on Vanilla for emitters (ARI 0.972, 5% of windows below 0.90) but does not match TOA-only rotary encoding. Rotating the deinterleaving branch by planar incidence is not, on this test set, a replacement for rotating every layer by TOA. Mode scores occupy a narrow band for Vanilla, RoPE-TOA, RoPE-az, and Bias. Bias is the tightest there, with the smallest count error.

RoPE-TOA+Bias does not keep the emitter gains of either parent. Mean emitter ARI is 0.853, below joint Vanilla, and 31% of windows fall below 0.90. Completeness is 0.851, so the leftover errors are splits. Mode ARI stays in the same band as the other joint models. Stacking the two time mechanisms is not an improvement on this split.

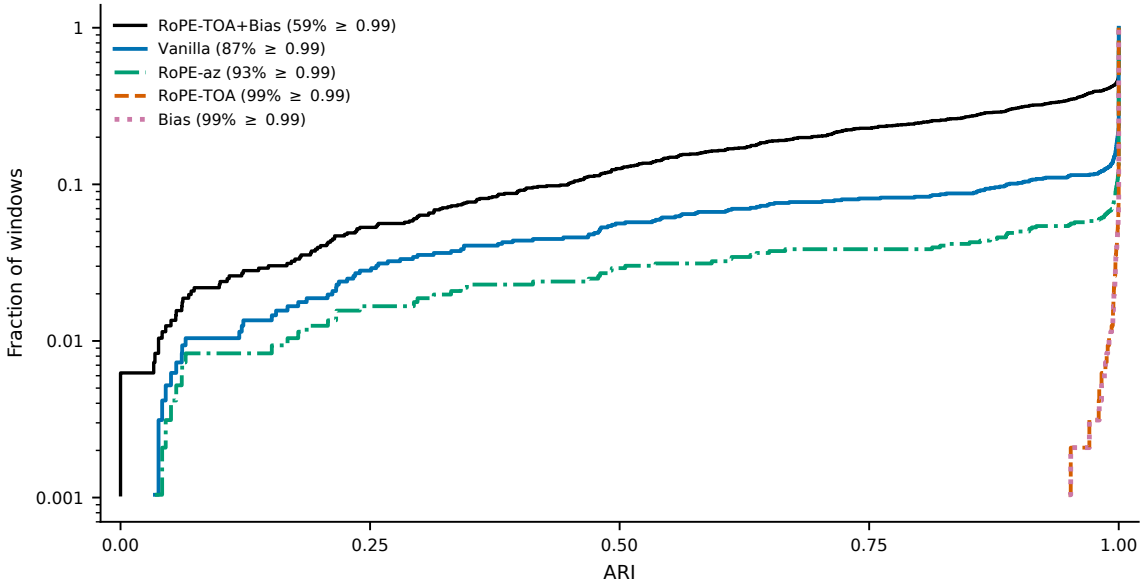


Figure 5.1. Empirical CDF of emitter ARI over the 960 non-overlapping test windows. The vertical axis is logarithmic.

5.6.1 Per-window ARI distributions

Figures 5.1 and 5.2 show the empirical CDF of per-window ARI for Vanilla, RoPE-TOA, RoPE-az, Bias, and RoPE-TOA+Bias. ARI is the metric used to select ε on the validation grid, so the curves correspond to the neural rows of Tables 5.6 and 5.7. Each curve is the fraction of the 960 windows whose ARI does not exceed the value on the horizontal axis. The vertical axis is logarithmic so that a tail of a few windows remains visible against the jump at 1. AMI, homogeneity, and completeness follow the same right spike and are not repeated as extra pages.

The Vanilla emitter curve leaves the axis well before 0.90. RoPE-TOA+Bias leaves earlier still, and 59% of its windows sit at or above 0.99 against 87% for Vanilla. RoPE-TOA and Bias stay near zero until just below 1, and the two curves nearly coincide on this split. RoPE-az sits between Vanilla and those two shapes. On the mode branch the five curves rise later and closer together. Bias is the last to leave zero. RoPE-TOA+Bias sits with Vanilla and RoPE-az.

5.6.2 Cluster-count recovery

Figures 5.3 and 5.4 are heatmaps of predicted versus true cluster counts. Each cell is the number of test windows with that pair. The diagonal is exact count recovery. Mass above the diagonal is extra clusters (splits). Mass below it is missing clusters (merges). That is why Tables 5.6 and 5.7 report homogeneity and completeness separately, and why they include a mean absolute count error.

On the emitter branch, Vanilla leaves the diagonal more often than RoPE-TOA, RoPE-az, and Bias, with splits and merges in roughly equal number. RoPE-TOA never

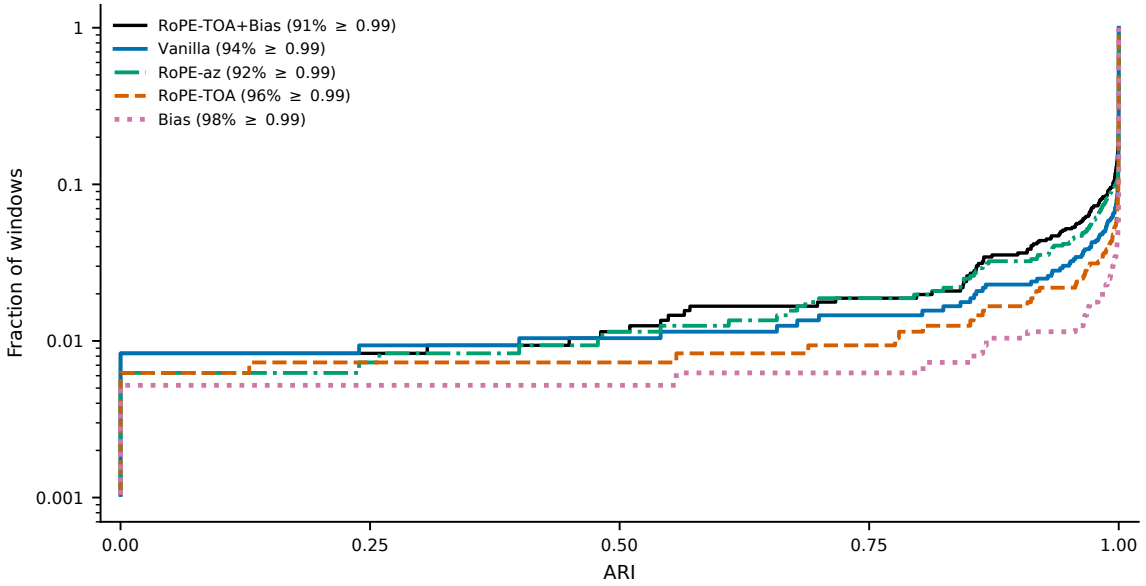


Figure 5.2. Empirical CDF of mode ARI over the same windows as Figure 5.1. The vertical axis is logarithmic.

oversplits an emitter window in this set: the off-diagonal mass is only undercounts. Bias is the same pattern, up to one window. RoPE-az is closer to the diagonal than Vanilla and still produces some extra clusters. RoPE-TOA+Bias leaves the diagonal more often than Vanilla, and the extra mass is above it.

The mode heatmaps are diagonal-dominant for every model. When they miss, they more often undercount than invent labels, and the misses concentrate at the crowded end of the catalogue, where a window can contain a dozen modes (Section 5.2). Bias keeps more mass on the diagonal than Vanilla or RoPE-az. RoPE-TOA+Bias is noisier there than Bias, with more off-diagonal counts at the high-M end.

5.6.3 Model size and inference cost

Table 5.8 reports trainable parameters and wall-clock time per test window on the A100 of Section 5.1. One window is forwarded at a time. A short GPU warmup is excluded from the mean. Clustering time is DBSCAN on the embeddings of Equation (4.14).

Rotary encoding adds no learned weights, so both RoPE models dump the same 16 081 536 parameters as Vanilla. That is fewer extra parameters than a learned positional table of length L, and fewer than the physical bias, which stores one scalar per head, layer, and selected coordinate. Those scalars do not change the dumped total at the reported precision. They do change the encoder time: Bias spends 30 ms in the forward pass where Vanilla spends 1.5 ms. RoPE-TOA+Bias is in the same band, 30.2 ms. The extra work is the pairwise differences $B^{(p)}_{ij} = |p_i - p_j|$ of Equations (4.16) and (4.17). Those matrices depend only on the window coordinates, so they are the same for every head and every attention sub-layer. The implementation rebuilds them

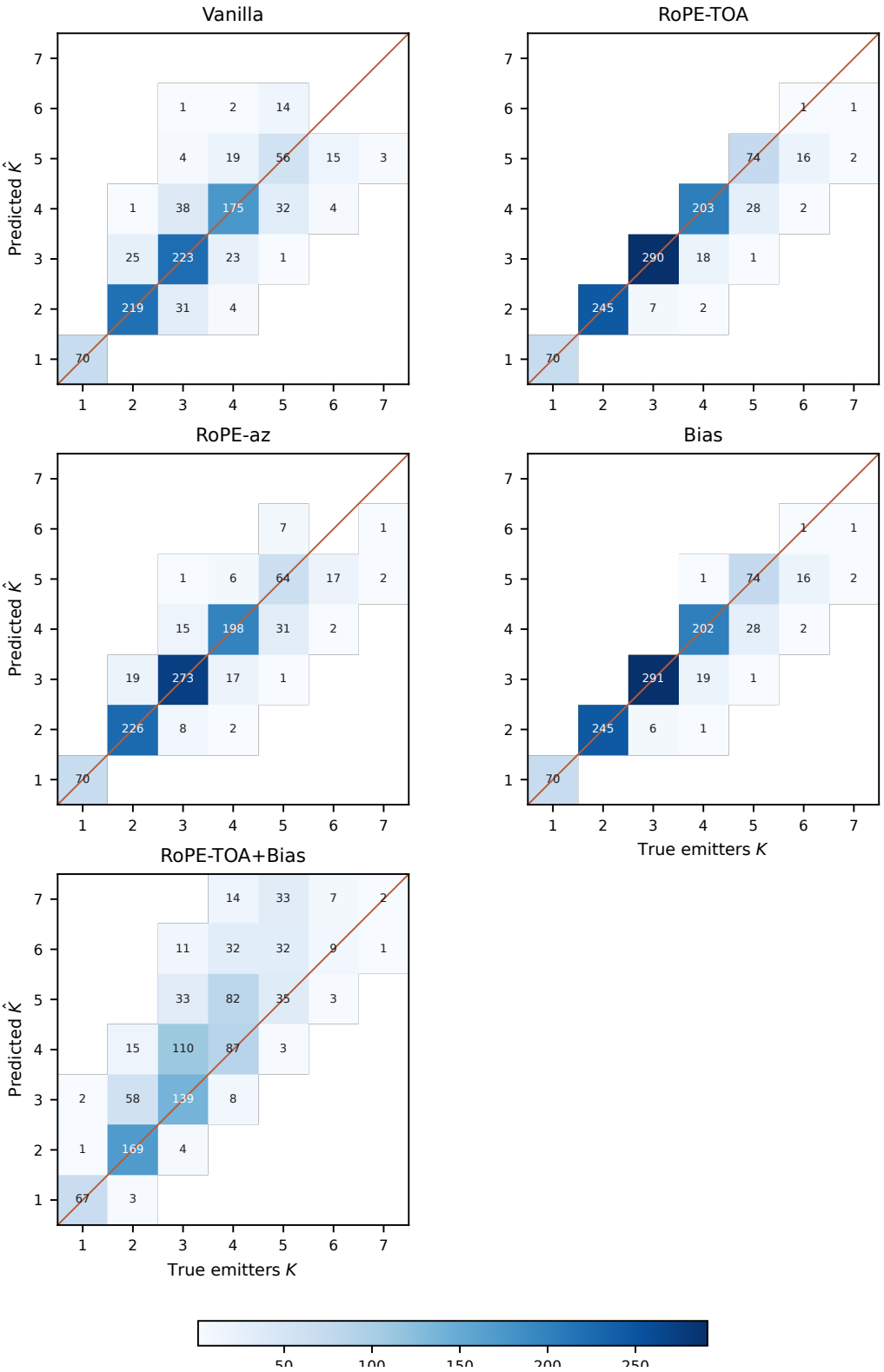


Figure 5.3. Predicted versus true number of emitters per test window. Cell values are window counts. The line is $\hat{K} = K$.

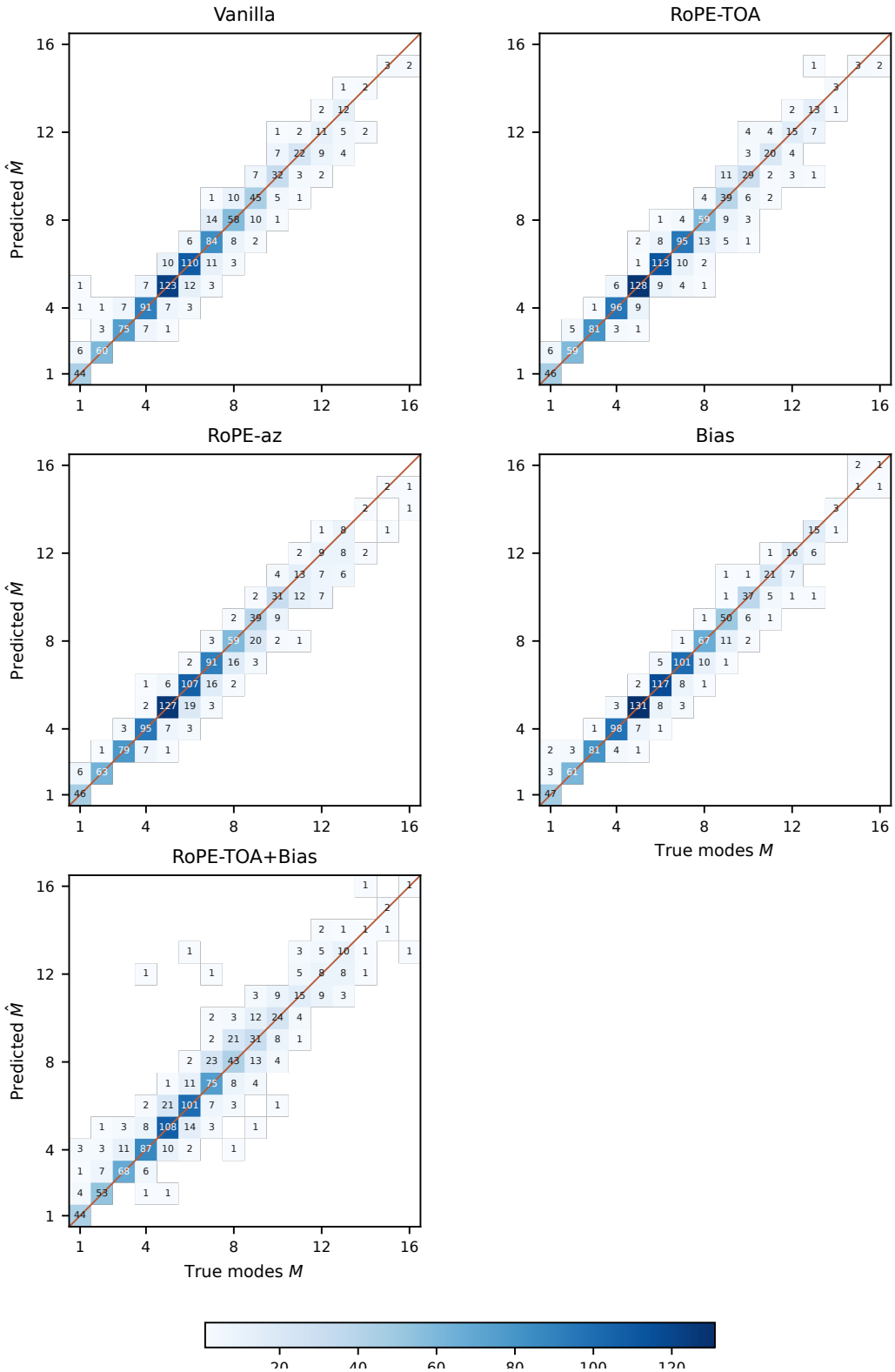


Figure 5.4. Predicted versus true number of modes per test window. Layout as in Figure 5.3.

Table 5.8. Trainable parameters and mean time per test window. Encoder is the forward pass, or the feature copy for Feature DBSCAN. Clustering is DBSCAN on both branches.

Model	Parameters	Encoder (ms)	Clustering (ms)	Total (ms)
Feature DBSCAN	0	0.1	20.9	21.0
Vanilla	16 081 536	1.5	61.2	62.7
RoPE-TOA	16 081 536	2.5	59.9	62.4
RoPE-az	16 081 536	2.9	61.3	64.2
Bias	16 081 536	29.6	62.0	91.6
RoPE-TOA+Bias	16 081 536	30.2	58.2	88.3

in each head of each sub-layer and then scales by that head's λ. A single copy per coordinate, computed once per window, would remove that repetition. No such cache was used. The time in Table 5.8 includes that redundant construction, not only the add into the logits. DBSCAN is about 60 ms for every neural model, so it dominates Vanilla and both RoPE variants. Feature DBSCAN has no encoder weights; its clustering time is 21 ms. Memory for the pairwise maps is taken up in Section 6.1.

5.6.4 Learned bias coefficients

Bias and RoPE-TOA+Bias store one scalar per head, layer, and selected coordinate of Equations (4.16) and (4.17). The trunk has only time. On the branches the checkpoint dump reports two scalars per head: a time coefficient λ_t and an incidence coefficient λ_{inc}. A negative value decreases the attention logit as the pairwise absolute difference grows, which is the sign used at initialisation.

Table 5.9 summarises the 32 scalars of each branch and coordinate (8 heads and 4 layers). The trunk mean stays near zero in both models. On the Bias mode branch, $\bar{\lambda}_t = 0.118$ and $\bar{\lambda}_{\text{inc}} = -0.877$. Every one of the 32 mode incidence scalars is negative. The deinterleaving branch has negative means of similar size on time and incidence (-0.202 and -0.229). Thirty of those 32 incidence scalars are negative. One head reaches $+0.832$ and inverts the cue.

RoPE-TOA+Bias keeps the same incidence pattern with smaller magnitude: mode $\bar{\lambda}_{\text{inc}} = -0.626$, again negative on every head, and deinterleaving means -0.167 (time) and -0.136 (incidence). Mode time coefficients that were positive under Bias become negative once rotary encoding already carries Δt.

5.7 Qualitative Analysis

The tables average over hundreds of windows. This section looks at the pulses themselves, first as input features and then as embeddings, to check that those averages correspond to a geometry a reader can inspect. The input is the same for every model, so the feature plots are shown once. Embeddings differ by model and by branch.

Table 5.9. Learned bias coefficients after training. Mean and range over 8 heads and 4 layers. The trunk has no incidence term. Negative values decrease attention as $|p_i - p_j|$ grows.

Model	Branch	$\bar{\lambda}_t$	$[\min, \max]_t$	$\bar{\lambda}_{\text{inc}}$	$[\min, \max]_{\text{inc}}$
Bias	Trunk	0.026	$[-0.240, 0.292]$	—	—
Bias	Mode	0.118	$[-0.596, 0.649]$	-0.877	$[-2.85, -0.016]$
Bias	Emitter	-0.202	$[-0.568, 0.110]$	-0.229	$[-0.528, 0.832]$
RoPE-TOA+Bias	Trunk	-0.037	$[-0.223, 0.132]$	—	—
RoPE-TOA+Bias	Mode	-0.107	$[-0.500, 0.195]$	-0.626	$[-2.17, -0.126]$
RoPE-TOA+Bias	Emitter	-0.167	$[-0.509, 0.042]$	-0.136	$[-0.296, 0.066]$

Input window

Figures 5.5 and 5.6 are the same $L = 2000$ pulses from evaluation window 100 (test scenario 1752). The four panels are carrier frequency, amplitude, pulse width, and a Cartesian incidence coordinate, each against window-local TOA. The export names that last panel Theta. It is a direction component after Equation (4.8), not a raw azimuth. Color is the only change between the two figures: Figure 5.5 uses the recording-local emitter label, Figure 5.6 the global mode label. Vanilla, RoPE-TOA, RoPE-az, Bias, and RoPE-TOA+Bias score ARI = 1 on both branches of this window, with two emitters and four modes.

The emitter coloring shows one platform with a scanning amplitude (the lower, scalloped track) and one platform whose amplitude sits on several flat levels. Frequency and pulse width already separate the scanner from the other platform, but they do not separate the flat tracks from each other: two of those tracks share a frequency band near 1.45 and the same narrow pulse width. Incidence does separate them. The mode coloring splits those flat tracks into different colors, which is the intended distinction. The scanner stays one color in both plots, so in this window that emitter uses a single mode.

Window 100 is a light mixture. Figure 5.7 is a crowded counterpart: an overlapping evaluation window with 17 distinct mode labels on four emitters, one more than the non-overlapping maximum used as M in Section 5.2. Frequency and pulse width collapse to a handful of horizontal bands that several modes share. Amplitude is no longer a set of flats. Several tracks scan or hop. Incidence still stacks platforms on separate levels, but the levels are closer and more numerous than in Figure 5.5. A method that clustered on one PDW coordinate would not recover this window. The dual-branch encoder still has to do so from the same four channels.

Embeddings

Figures 5.8 and 5.9 are UMAP projections of the per-pulse embeddings on window 100. Each panel is one model. Color is again the ground-truth label of that branch (two emitters, four modes). The source plots have no legend. The two-color panels match

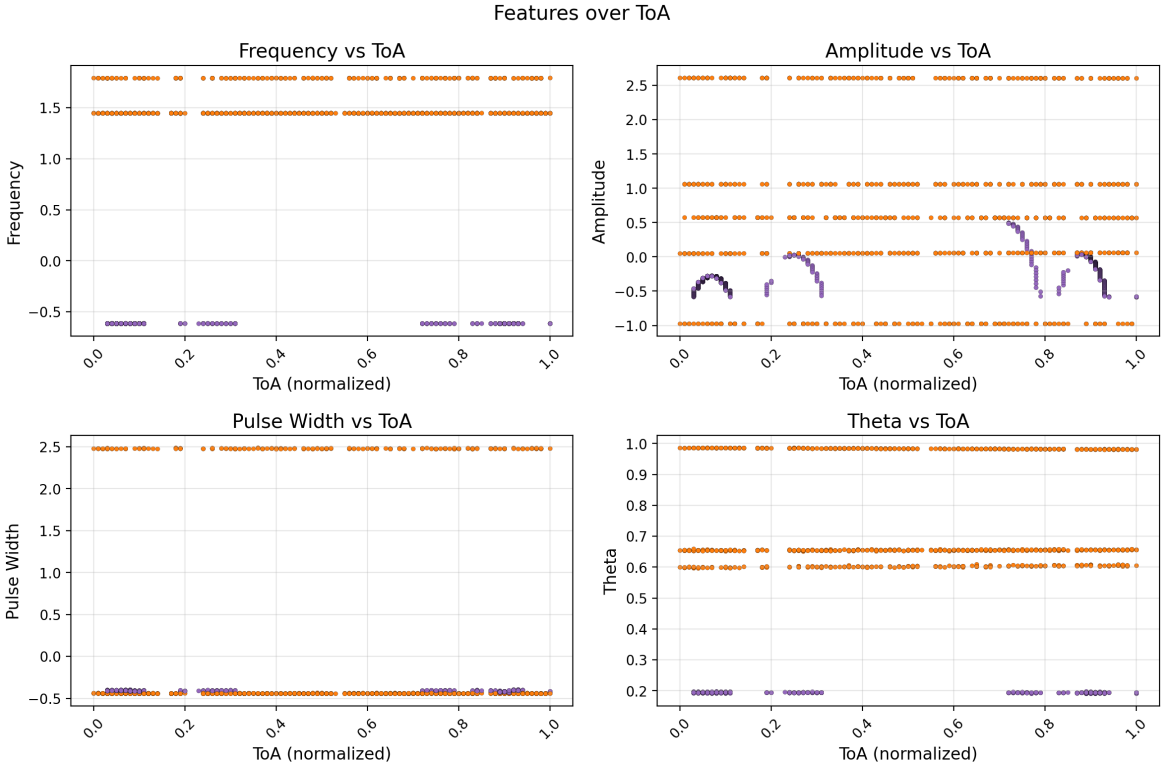


Figure 5.5. Evaluation window 100, colored by ground-truth emitter. Two emitters are present.

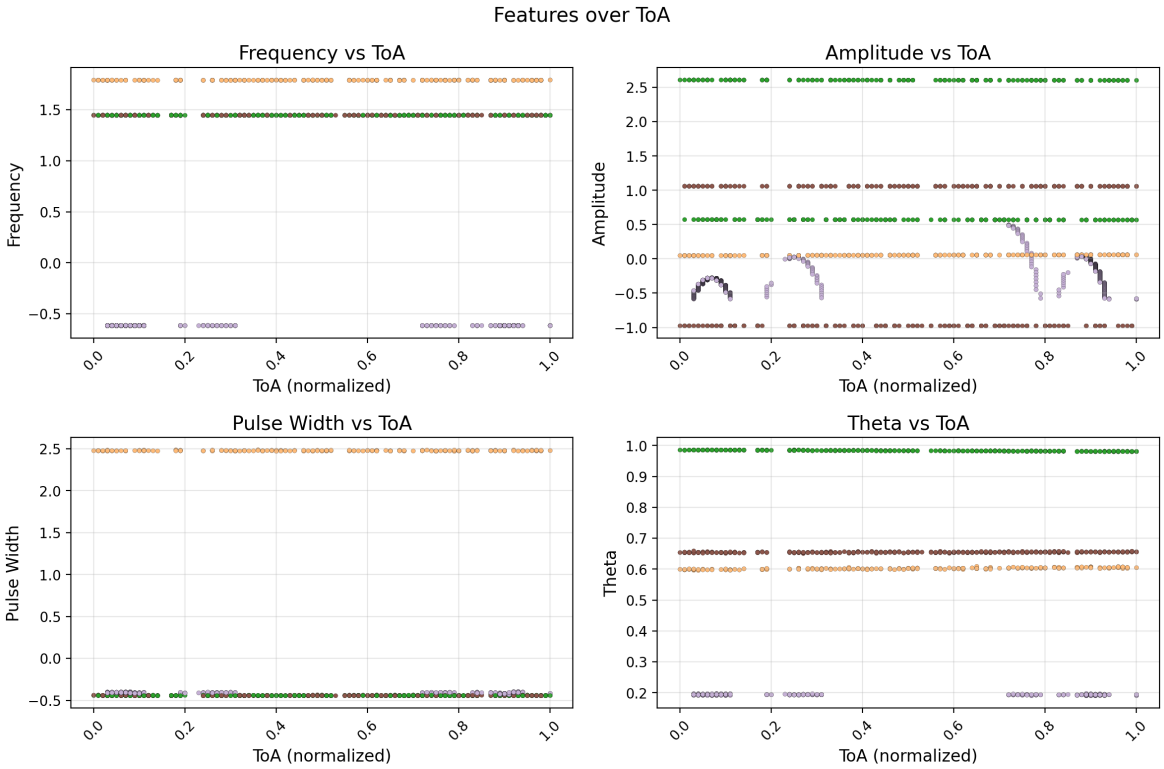


Figure 5.6. The same pulses as Figure 5.5, colored by ground-truth mode. Four modes are present.

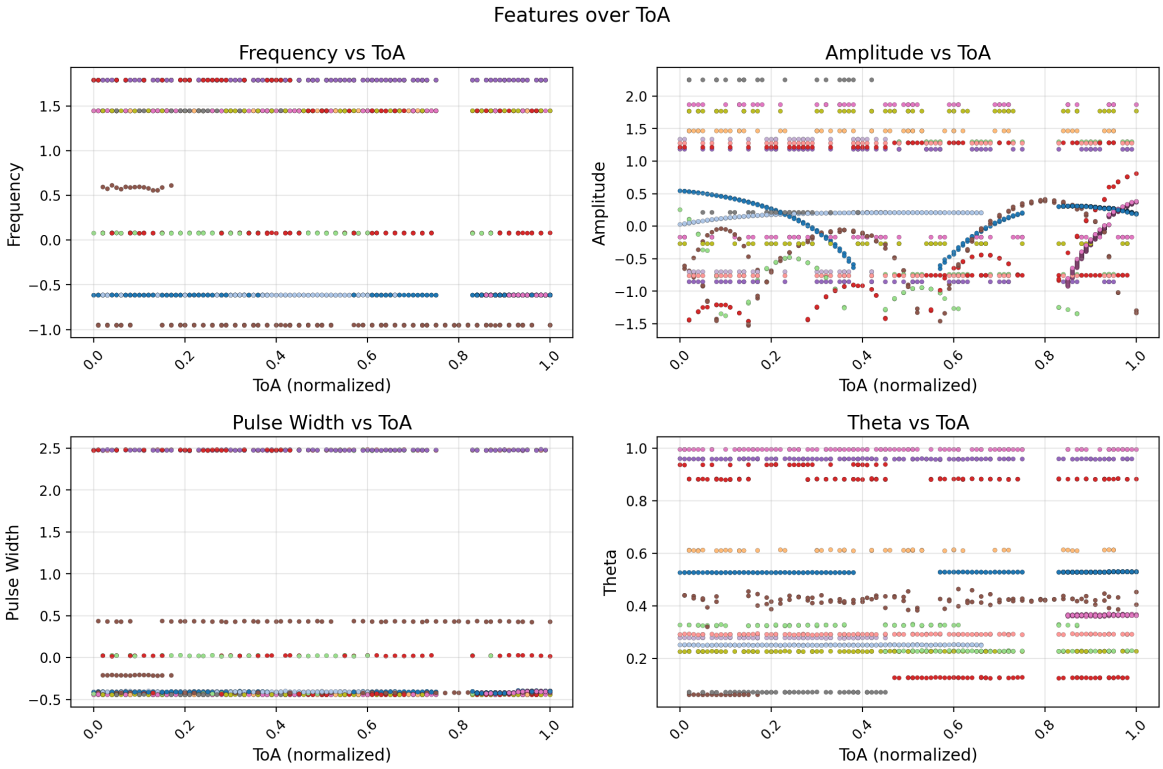
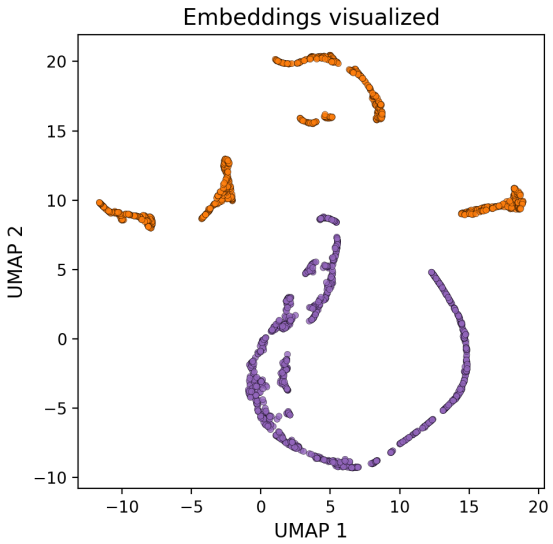


Figure 5.7. A crowded overlapping test window (evaluation index 4126), colored by ground-truth mode.

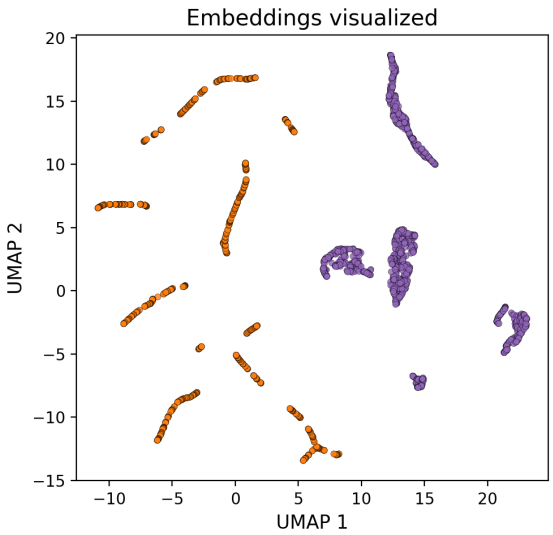
Figure 5.5, the four-color panels match Figure 5.6. The projection is used only to look at the embeddings. It is not part of decoding.

On the emitter branch the two colors occupy separate regions for every model. Vanilla keeps one emitter as several small islands and the other as a connected, more spread component, which matches the scanner versus the multi-level platform in the features. RoPE-TOA and Bias pull those islands tighter and leave a wider gap between the two colors. RoPE-az remains closer to Vanilla: the two colors still split, but the layout is less compact. RoPE-TOA+Bias also splits the two colors on this window. The orange side is a set of thin arcs, closer to RoPE-TOA than to Vanilla. This window is not where that model fails. The splits in Table 5.6 and Figure 5.3 sit elsewhere.

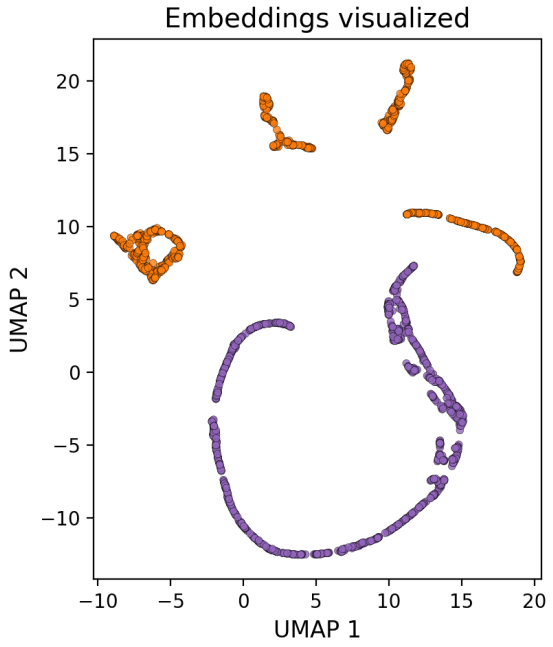
On the mode branch the four colors separate in every panel. Some colors form thin arcs rather than round blobs, which is consistent with a mode whose amplitude or timing still moves inside the window. None of the five joint models disagrees on this window in ARI, so the visual differences are layout, not a change in the discrete partition.



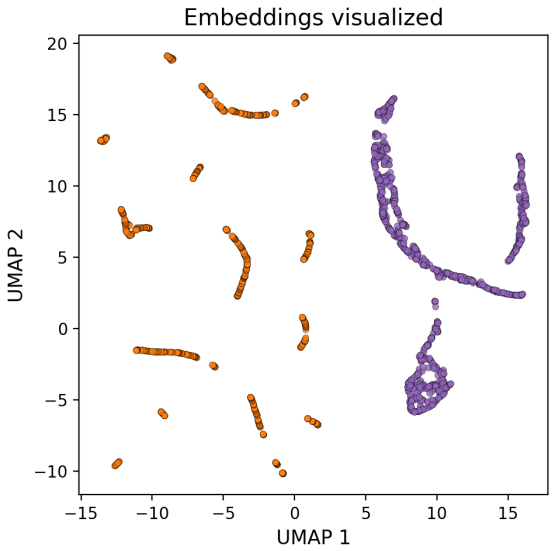
(a) Vanilla



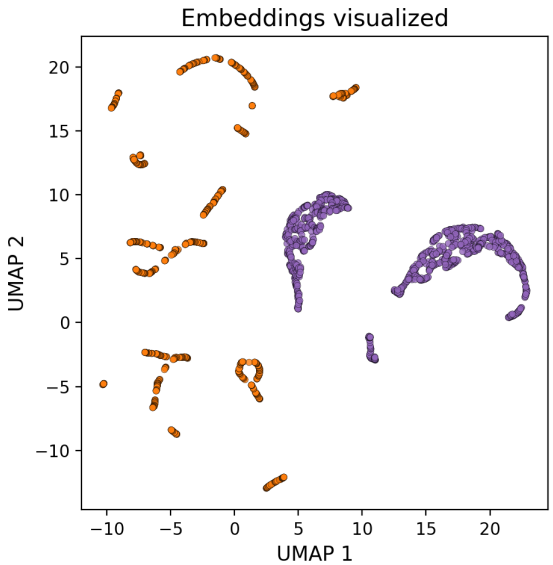
(b) RoPE-TOA



(c) RoPE-az

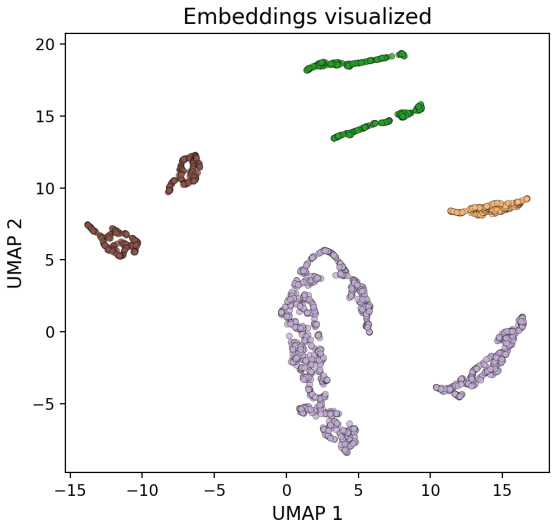


(d) Bias

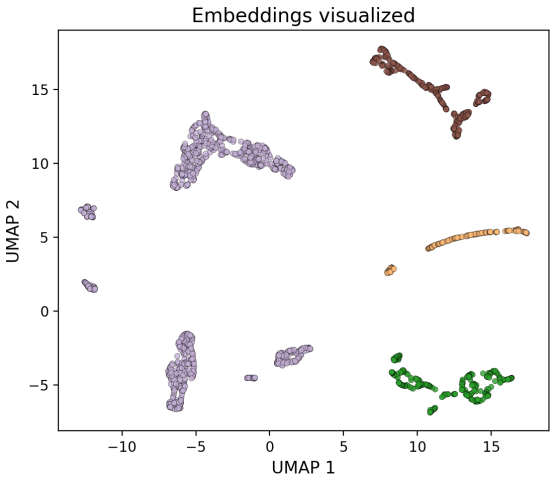


(e) RoPE-TOA+Bias

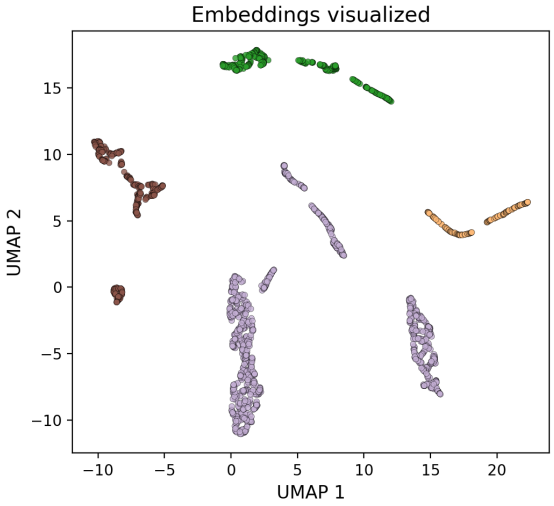
Figure 5.8 UMAP of emitter-branch embeddings on window 100, colored by ground-truth



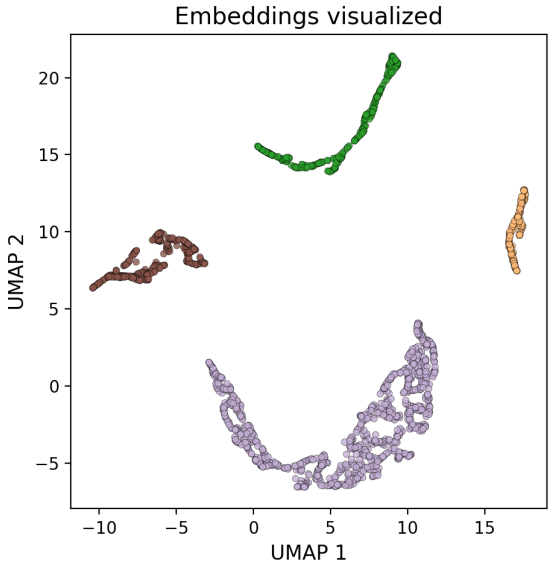
(a) Vanilla



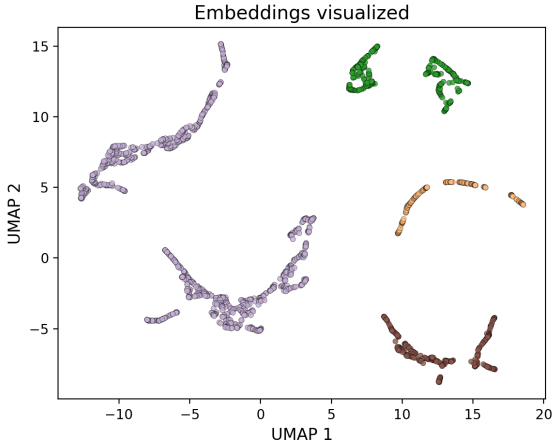
(b) RoPE-TOA



(c) RoPE-az



(d) Bias



(e) RoPE-TOA+Bias

Figure 5.9. UMAP of mode-branch embeddings on window 100, colored by ground-truth mode.

CHAPTER 6

Discussion

6.1 Interpretation of Results

Mode clustering is already strong with standard self-attention. The attention variants mainly move the emitter scores, and they do not all move them the same way.

Vanilla's leftover emitter errors sit in a minority of windows rather than on every window. The tail in Figure 5.1 and the off-diagonal mass in Figure 5.3 are those windows: mixtures in which frequency and pulse width are shared across platforms, of the kind drawn in Figure 5.7. Window 100 is the opposite case. The two emitters are already separated in incidence and in the scanning amplitude, and every jointly trained model in Figure 5.8 recovers both partitions.

RoPE-TOA puts time into every attention product. On this corpus that is enough to close the emitter tail, with no extra weights. Inter-arrival structure is the cue that distinguishes pulse trains once RF and pulse width collide, and rotating queries and keys by window-local TOA makes that cue available in the inner product rather than only as one input channel.

RoPE-az spends the deinterleaving rotations on planar incidence instead. Incidence is a useful input feature, as Figure 5.5 shows, but using it as the rotary coordinate of the deinterleaving branch does not beat rotating that branch by TOA. The embeddings in Figure 5.8 agree with the table. RoPE-az still splits the two colors, and it does not tighten them the way RoPE-TOA does. A likely reason is that the Cartesian incidence channels are already in the token. Rotary encoding of those channels restates a cue the encoder already has, while TOA rotary encoding restates a cue that standard attention otherwise has to learn from a single scalar.

The physical bias reaches the same emitter scores as RoPE-TOA by a more expensive route. It adds a handful of learned scalars, not a second network. What it adds at run time is a pairwise map of shape $h \times L \times L$ per layer. With $h = 8$ and $L = 2000$, one such map is 128 MB in fp32. Twelve attention sublayers make about 1.5 GB if the maps are all resident, and a training dump at batch size 4 reports 30 GB of forward and backward storage for Bias against 19 GB for Vanilla. That gap is the pairwise tensors, counted twice when gradients are kept. At the evaluation batch size of one window the extra is a few gigabytes rather than tens, which matches the encoder time in Table 5.8: 30 ms instead of 1.5 ms, while DBSCAN stays near 60 ms for every model. The 30 ms includes the uncached rebuild of the same $|p_i - p_j|$ matrices in every head of every sub-layer (Section 5.6.3). A cache would remove the rebuild. It would not remove the add into each head's logits, which is still quadratic in L.

The intended split of Section 4.4.1 was that deinterleaving would use incidence and that the mode branch would ignore it. Table 5.9 does not show that split. The largest coefficients sit on the mode branch, and they are incidence, not time. A negative λ_{inc} is a same-direction gate: pulses that differ in arrival direction attend less. That is a platform cue. It can still help a mode that occupies one emitter inside a window, and it can hurt a mode that is seen from two directions. The Bias mode scores remain the tightest in Table 5.7, so the gate does not break mode clustering on this corpus. Deinterleaving uses time and incidence at about the same scale, which is both cues rather than an incidence-only head. Trunk coefficients stay small, so most of the pairwise work is in the branches.

RoPE never builds those maps. Its dumped parameter count matches Vanilla, and its encoder time stays within a couple of milliseconds of Vanilla. The total-size figure printed for the RoPE summaries is an artifact of expanding rotation blocks in the layer dump, not a measured allocation.

Stacking the two time mechanisms does not add their gains. RoPE-TOA+Bias pays the Bias encoder time and scores below Vanilla on emitters (Table 5.6 and Figure 5.1). The extra errors are splits (Figure 5.3). Both devices already put a function of Δt into the attention product, so the stacked logits can double-count time. The stacked checkpoint still has a large negative mode incidence coefficient (Table 5.9), while its mode time coefficients change sign relative to Bias. That is consistent with rotary encoding already carrying time and with the bias remaining an incidence gate. The tables give no reason to pay for both.

The accuracy of the bias is a statement about pairwise geometry. A few scalars on $|p_i - p_j|$ recover the same emitter partition as rotating every query and key by time. Vanilla has to learn that structure from a single TOA channel. The differences themselves already carry it. That points to a network whose layers read those delta matrices, rather than adding them onto a learned query–key product. Self-attention on a window is already mixing on a complete graph (Section 2.7). The bias is already the Graphormer-style edge term of Sections 3.4 and 4.4.1. A graph network whose edges carry $(|\Delta t|, |\Delta \tilde{u}^x|, |\Delta \tilde{u}^y|)$ would use that pairwise physics directly. Whether it matches the scores, and whether it is cheaper once the differences are computed once, is not measured here.

Joint training is a separate axis. Emitter-only Vanilla already deinterleaves more cleanly than joint Vanilla, while its untrained mode branch collapses by merging (Table 5.5). Mode-only is the mirror image: the mode scores match Joint, and the untrained emitter branch splits. A trunk trained only for modes has a reason to separate pulses that share an emitter but not a mode. The emitter head, with no loss of its own, inherits that split. The two contrastive terms pull on incompatible similarities in the shared trunk: pulses that should be close as modes may need to be far as emitters. At $\lambda_{\text{md}} = 1$ the trunk yields to that conflict on the emitter side and keeps the mode side intact. Joint training is still the only objective in the table that leaves both

partitions usable. The attention variants were all trained jointly, so the emitter gains of RoPE-TOA and Bias are gains under that more demanding objective, not under a deinterleaving-only loss. Whether those variants would still separate from Vanilla if trained emitter-only is not measured here.

6.2 Comparison with Related Work

Classical ELINT pipelines surveyed in Section 3.1 deinterleave with TOA histograms or with density clustering on raw PDW coordinates, then match trains against a library. The encoder here replaces that first stage with a learned embedding, and it never performs the second. Scores in Chapter 5 are partition agreement, not equipment names. A raw-feature DBSCAN control is the Feature DBSCAN rows of Table 5.4. It oversplits emitters. Vanilla is already well above that control, so the encoder is doing work that density clustering on the same scaled PDWs does not. On windows where frequency and pulse width collide, as in Figures 5.5 and 5.7, a metric on those coordinates alone is not enough. The learned embeddings still separate the platforms.

Supervised radar classifiers in Section 3.2 assume a global emitter catalogue and a softmax at inference. Jonsson [19] trains a related contrastive encoder on the same simulator lineage, still as closed-set identification. The present scores cannot be stacked against that accuracy. The tasks differ. Emitter labels here are recording-local, and both branches are clustered rather than classified. That is closer to Gunn et al. [21], who train a transformer for deinterleaving and cluster the embeddings with HDBSCAN. The Vanilla emitter row of Table 5.4 is the analogous existence test, on a different corpus and with DBSCAN rather than HDBSCAN. Gunn et al. stop at one partition. The Vanilla mode row of Table 5.4 is the addition, and Table 5.5 shows that it is not free: joint Vanilla deinterleaves less cleanly than an emitter-only copy of the same encoder, and a mode-only copy splits emitters.

The attention variants go one step past that counterpart. Gunn et al. use standard dot-product attention and leave TOA as an input feature. RoPE-TOA and Bias put time into the attention product itself, and on this test set they close the emitter tail that Vanilla leaves. RoPE-az does not, once incidence is already in the token. Stacking the two time mechanisms does not close the tail either. None of these variants is a published radar baseline. They are drop-in changes of the same encoder, which is the comparison RQ3 asks for.

Deep clustering methods in Section 3.3 usually learn one partition, often with the number of clusters chosen in advance. The two branches here are not a hierarchy of one relation. Modes are not subtypes of emitters. A single embedding cannot keep both similarities, and Table 5.5 is the empirical trace of that conflict in the shared trunk: the unused head either merges or splits. The comparison with multi-task softmax heads in Section 3.5 is the same point in other words. Those heads need globally consistent class names. The emitter identifiers in this corpus are not that.

6.3 Limitations

All tabulated scores come from one synthetic corpus with full per-pulse labels. The code is proprietary and is not released with this thesis.

The test modes are the training modes. Held-out modulation types were not scored, even though clustering rather than a softmax is what would make that experiment possible. Claims about novel emitters or novel modes at inference therefore go beyond the tables. Emitter identifiers are recording-local by construction, which is the right operational assumption, but it is not a measured open-set test.

Every score is a window score. The encoder never sees a full recording in one forward pass, trailing segments shorter than L are dropped, and nothing is tracked across window boundaries. Self-attention and the physical bias both scale as L^2. Extending the window would raise the memory gap in Section 6.1 and would still not be streaming inference.

Pulse counts per label are unbalanced inside windows, as Section 5.2 notes. Rare modes and sparse emitters contribute little to a window-mean ARI. A method can look strong in Table 5.7 while still failing the labels that occupy few pulses. The crowded window in Figure 5.7 is a reminder of that regime, not a substitute for a stratified breakdown.

λ_{md} was not swept. The finding that joint training costs emitter ARI on Vanilla is therefore tied to equal loss weights and to that backbone. Whether RoPE-TOA or Bias would show the same cost under an emitter-only objective is unknown.

DBSCAN is part of the reported system. The radius is chosen on the validation grid, minPts is fixed, and the noise category was ignored because it stayed small on this corpus. A different density clusterer, or a different ε rule, would change the discrete partitions without changing the embeddings.

Finally, no operational ELINT recordings enter the numbers. Simulator drop and spurious-pulse models exist in the generator, but they are not the test set behind the tables. Distribution shift, library mismatch, and deinterleaving errors of the kind studied by Jakobsson [37] and Coleman [20] remain outside the present scores.

6.4 Implications

If the task is deinterleaving alone, the Vanilla ablation is the practical message. Train the emitter term only. If the task is modes alone, train the mode term only. The joint loss is for the case where both partitions are required from the same window, and even then the equal-weight sum is a compromise rather than a free extra head.

If both partitions are required, standard self-attention is already enough for modes on this corpus and is not enough for the hardest emitter windows. Putting TOA into the attention product, either as RoPE-TOA or as the pairwise bias, is what closes that gap. RoPE-TOA does it without extra weights and without a material change in encoder time. The bias matches those scores and spends memory and milliseconds on

pairwise maps. Using both at once does not help on this split, and it keeps the pairwise cost. Incidence rotary encoding on the deinterleaving branch is not a substitute, once the Cartesian direction channels are already input features.

The clustering path at inference keeps the output open in form. It does not, by itself, demonstrate that unseen modes would form their own clusters. Until a held-out-modulation split is scored, operators should read the mode indices as a grouping of catalogue types that were present in training, not as a detector of new waveforms. Emitter indices are window-local hypotheses. They still need a later association stage if the product is a track, not a window.

For representation learning more broadly, the useful fact is the one in Table 5.5. Two supervised contrastive terms with different positive-set scopes can share a trunk, and the trunk will trade one partition against the other. That is expected once the two similarities conflict, and it is measurable. Tuning λ_{md}, or detaching more of the trunk, is the obvious next control when both heads must stay close to their single-task ceilings.

The ethical limits in Section 1.6 do not change with these scores. The work is a clustering method on synthetic pulse features. It is not an identification system, and it is not ready to be read as one.

CHAPTER 7

Conclusion

7.1 Summary

This thesis asked whether a transformer encoder can recover two partitions from one interleaved PDW window: emitters that are comparable only inside a recording, and operating modes that are comparable across recordings. The encoder shares a trunk, splits into two branches, and is trained with two supervised contrastive losses that differ only in the scope of their positive sets. Inference clusters each branch with DBSCAN. No catalogue softmax is used.

On the synthetic test split, standard self-attention already recovers both partitions on most windows. Rotary encoding of window-local TOA, and a pairwise physical bias on the same coordinates, close the remaining emitter errors. Using both together does not. A joint loss is not a free improvement over a single-task encoder. It costs emitter agreement relative to deinterleaving-only training, it splits emitters relative to mode-only training, and it is what keeps both branches usable.

7.2 Answers to the Research Questions

RQ1 Yes, on this corpus. The Vanilla encoder of Table 5.4 deinterleaves pulses and recovers modes by clustering a bounded window of PDWs. Mode agreement is high on almost every window. Emitter agreement is high on most windows and fails on a minority of crowded mixtures. Both scores sit well above Feature DBSCAN on the same test split.

RQ2 Not on the emitter task. A Vanilla encoder trained only on $\mathcal{L}_{\text{em}}$ outperforms joint Vanilla on emitter ARI (Table 5.5). A Vanilla encoder trained only on $\mathcal{L}_{\text{md}}$ matches Joint on modes and splits emitters. The joint objective is the setting in which both partitions remain usable.

RQ3 Yes for time, with a cost that depends on how time is injected. RoPE-TOA and the physical bias both remove the Vanilla emitter tail (Table 5.6 and Figure 5.1). Stacking them does not: RoPE-TOA+Bias is slower than Vanilla and worse on emitters. Rotary encoding of planar incidence on the deinterleaving branch improves on Vanilla and does not match TOA-only rotary encoding. Mode scores move much less. The bias matches RoPE-TOA on accuracy and spends extra memory and encoder time on pairwise maps (Table 5.8 and Section 6.1).

7.3 Future Work

A sweep of λ_{md} would show whether the emitter cost of the joint loss can be reduced without giving the mode branch back to merging or splitting. Retraining the attention variants under an emitter-only objective would show whether RoPE-TOA and Bias still separate from Vanilla when the trunk is not also serving modes. A stack that puts RoPE on time and the bias only on incidence would test whether the stacked failure is double-counting of Δt.

Held-out modulation types are the experiment that the clustering decoder is for. Until that split is scored, the mode branch is a grouping of seen catalogue types. Operational recordings, and simulator drop or spurious-pulse regimes as an actual test set, are the corresponding check on distribution shift.

The scores in Chapter 5 describe a window, not a recording. Cluster indices stop at the window boundary, so the same emitter in two adjacent windows is not known to be the same object. A cheap association is to replace each window cluster by the mean of its embeddings and to cluster those means over the recording with DBSCAN. The pulse encoder stays frozen. A second transformer on top of those means, or of the window embeddings, could learn the association instead of leaving it to a second density step. Either construction yields one partition per scenario.

Mode labels are already global. A classification head on the mode embeddings would name types from the training catalogue, while clustering remains the decoder for types that are not in that list. The head can sit on a frozen encoder, or it can be trained jointly. The clustering scores in Table 5.7 do not say whether the catalogue is linearly separable in that space.

Mean ARI hides which recordings are hard. Simple corpus statistics would make that visible: how often two emitters share a frequency band, how uneven the pulse counts are, how many modes sit in one window, and how large the TOA gaps are relative to the PRI of each mode. Those quantities can be computed from the simulator labels without a new model. They would also make a later comparison to another generator, or to operational data, a comparison of the same axes rather than of two unlabelled averages.

Only DBSCAN is used at inference. HDBSCAN would remove the ε grid. A Gaussian mixture or spectral clustering would assume a different geometry in the same embeddings. Running those decoders on frozen embeddings would separate encoder error from decoder error.

The encoder is bidirectional over a block of L pulses. A live stream does not arrive as a block. Keys and values from pulses already in the window can be cached, so a new pulse attends against that cache instead of forcing a full forward pass. A mask then chooses which cached pulses remain visible: a sliding window of length L, or a causal band if later pulses must not leak into the present. That is incremental inference. Updating the weights from the stream is a separate problem and is not required for the

cache.

References

[1] R. G. Wiley, *ELINT: The Interception and Analysis of Radar Signals*. Artech House, 2006.

[2] M. I. Skolnik, *Radar Handbook*, 3rd ed. McGraw-Hill, 2008.

[3] Z. Qu, J. Zhang, Y. Zhou, and L. Ni, "The intelligent evolution of radar signal deinterleaving: A systematic review from foundational algorithms to cognitive AI frontiers", *Sensors*, vol. 26, no. 1, 2026. DOI: 10.3390/s26010248

[4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[5] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis", in *International Conference on Machine Learning (ICML)*, 2016, pp. 478–487.

[6] L. Hubert and P. Arabie, "Comparing partitions", *Journal of Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[7] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance", *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[8] A. Rosenberg and J. Hirschberg, "V-measure: A conditional entropy-based external cluster evaluation measure", in *Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL)*, 2007, pp. 410–420.

[9] A. van den Oord, Y. Li, and O. Vinyals, *Representation learning with contrastive predictive coding*, arXiv preprint arXiv:1807.03748, 2018. arXiv: 1807.03748 [cs.LG].

[10] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations", in *International Conference on Machine Learning (ICML)*, 2020.

[11] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding", in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[14] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding", *Neurocomputing*, vol. 568, p. 127 063, 2024. DOI: 10.1016/j.neucom.2023.127063

[15] H. K. Mardia, "New techniques for the deinterleaving of repetitive sequences", *IEE Proceedings F (Radar and Signal Processing)*, vol. 136, no. 4, pp. 149–154, 1989. DOI: 10.1049/ip-f-2.1989.0025

[16] D. J. Milojević and B. M. Popović, "Improved algorithm for the deinterleaving of radar pulses", *IEE Proceedings F (Radar and Signal Processing)*, vol. 139, no. 1, pp. 98–104, 1992. DOI: 10.1049/ip-f-2.1992.0012

[17] S. Bedoire, "Offline direction clustering of overlapping radar pulses from homogeneous emitters", Host company Saab; DBSCAN-style clustering on pulse directions with simulated scenarios, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2022.

[18] P. Notaro, M. Paschali, C. Hopke, D. Wittmann, and N. Navab, *Radar emitter classification with attribute-specific recurrent neural networks*, arXiv preprint arXiv:1911.07683, 2019. arXiv: 1911.07683 [eess.SP].

[19] T. Jonsson, "Classification of Radar Emitters using Semi-Supervised Contrastive Learning", Host company Saab AB; supervised learning experiments on pulse data from the OpenModesty scenario simulator, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2023.

[20] K. Coleman, "Dataset drift in radar warning receivers: Out-of-distribution detection for radar emitter classification using an RNN-based deep ensemble", UPTEC F 23041, 30 hp; simulator data from Saab AB and drift/OOD analysis for emitter classification, Master's thesis, Uppsala University, Uppsala, Sweden, 2023.

[21] E. Gunn, A. Hosford, D. Mannion, J. Williams, V. Chhabra, and V. Nockles, *Radar pulse deinterleaving with transformer based deep metric learning*, arXiv preprint arXiv:2503.13476, 2025. arXiv: 2503.13476 [cs.LG].

[22] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection", *ACM Transactions on Knowledge Discovery from Data*, vol. 10, no. 1, 2015. DOI: 10.1145/2733381

[23] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering", *The Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017. DOI: 10.21105/joss.00205

[24] X. Guo, L. Gao, X. Liu, and J. Yin, "Improved deep embedded clustering with local structure preservation", in *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI)*, 2017, pp. 1753–1759. DOI: 10.24963/ijcai.2017/243

[25] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features", in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 132–149.

[26] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin, "Unsupervised learning of visual features by contrasting cluster assignments", in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[27] Y. Li, P. Hu, Z. Liu, D. Peng, J. T. Zhou, and X. Peng, "Contrastive clustering", in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 8547–8555. DOI: 10.1609/aaai.v35i10.17037

[28] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823. DOI: 10.1109/CVPR.2015.7298682

[29] J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks", in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, PMLR, 2019, pp. 3744–3753.

[30] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning", in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2021, pp. 2114–2124. DOI: 10.1145/3447548.3467401

[31] P. Shaw, J. Uszkoreit, and A. Vaswani, "Self-attention with relative position representations", in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2018, pp. 464–468. DOI: 10.18653/v1/N18-2074

[32] C. Ying, T. Cai, S. Luo, S. Zheng, G. Ke, D. He, Y. Shen, and T.-Y. Liu, "Do transformers really perform bad for graph representation?", in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 28 877–28 888.

[33] B. Heo, S. Park, D. Han, and S. Yun, "Rotary position embedding for vision transformer", in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024, pp. 289–305. DOI: 10.1007/978-3-031-72684-2_17

[34] R. Caruana, "Multitask learning", *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997. DOI: 10.1023/A:1007379606734

[35] J. Yang, D. Parikh, and D. Batra, "Joint unsupervised learning of deep representations and image clusters", in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5147–5156.

[36] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization", in *International Conference on Learning Representations (ICLR)*, 2019.

[37] V. Jakobsson, "Offline radar pulse track association with deinterleaving errors", Host company SAAB; time-series clustering for track fragments under imperfect deinterleaving, Master's thesis, KTH Royal Institute of Technology, Stockholm, Sweden, 2024.

APPENDIX A

Probabilistic Reading of the Contrastive Loss

This appendix expands the supervised contrastive kernel of Equation (4.25). The main text states the loss used in training; the derivation below is optional background.

For each pulse $i \in \mathcal{B}^+$, define the categorical distribution over its candidates

$$P_{\boldsymbol{\theta}}(a \mid i) \;=\; \frac{\exp(s_{ia}/\tau)}{Z_i}, \qquad Z_i \;=\; \sum_{a' \in A(i)} \exp(s_{ia'}/\tau), \qquad a \in A(i), \tag{A.1}$$

which is the Boltzmann distribution with energy $-s_{ia}$ and temperature τ on $A(i)$. Substituting Equation (A.1) into Equation (4.25) yields the per-pulse cross-entropy between the empirical positive distribution (uniform on $P(i)$) and the model $P_{\boldsymbol{\theta}}(\cdot \mid i)$,

$$\mathcal{L}_{\mathrm{SC}} \;=\; -\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i). \tag{A.2}$$

That is minus the average log-evidence assigned to the positives, as if $P(i)$ were observations from a latent positive class.

Applying $\log(x/y) = \log x - \log y$ to Equation (4.25) separates attraction and repulsion:

$$\mathcal{L}_{\mathrm{SC}} \;=\; \underbrace{\sum_{i \in \mathcal{B}^+} \log Z_i}_{\text{log-partition (repulsion)}} \;-\; \underbrace{\sum_{i \in \mathcal{B}^+} \frac{1}{|P(i)|} \sum_{p \in P(i)} \frac{s_{ip}}{\tau}}_{\text{mean positive similarity (attraction)}} . \tag{A.3}$$

The first term grows with similarity to every candidate; its gradient $\partial \log Z_i / \partial s_{ia} = P_{\boldsymbol{\theta}}(a \mid i)/\tau$ concentrates on hard negatives, i.e. candidates with large Boltzmann weight under Equation (A.1) despite a different label. The second term rewards alignment with the positives, each contributing $1/(\tau|P(i)|)$ to the gradient.

The prefactor $1/|P(i)|$ turns the inner sum into an arithmetic mean, or equivalently the log of a geometric mean,

$$-\frac{1}{|P(i)|} \sum_{p \in P(i)} \log P_{\boldsymbol{\theta}}(p \mid i) \;=\; -\log \left(\prod_{p \in P(i)} P_{\boldsymbol{\theta}}(p \mid i) \right)^{1/|P(i)|} . \tag{A.4}$$

Without it, the inner sum would be the log-likelihood of $|P(i)|$ independent positive observations and would scale with the size of $P(i)$. Dividing by $|P(i)|$ makes each pulse's contribution intensive in its positive-set size. That is why the same kernel stays

well-behaved on both branches, where $|P(i)|$ ranges from a few emitter matches inside one window to many hundreds of mode matches across the batch.